



通过即时智能体支架演化扩展支架智能

LV-NUS Lab

摘要

智能体能力并非仅由模型决定。智能体支架涵盖记忆管理、规划策略、动作协议以及工具/技能编排，其作用可能压过底层基础模型本身的贡献。然而，支架设计仍依赖人工、针对特定任务，因而从根本上难以扩展。我们提出 JIT-Agent：一种支架智能模型，经过训练后可为任意现成的智能体大语言模型即时合成适应任务的智能体支架。我们将智能体支架形式化为一种可组合、可由机器生成，并受固定四模块协议约束的工件；随后训练 JIT-Agent 针对手头任务定制支架，为稳定可靠的执行修复支架，并从不断扩充的既往支架配置档案中提炼性能信号以实现自我演化。在 JIT-Agent 作为支架助手的情况下，DeepSeek-V4-Flash 在 DeepSearchQA 和 OdysseyBench 上分别超过 GPT-5.6 9.1 和 4.3 分，而本已强大的 GLM-5.2 最多还能提升 20.2 分。在受控评测中，JIT-Agent 生成的支架在性能上可与 OpenCode、Claude Code 等成熟智能体运行时竞争，并能持续改善 DeepSeek V4、Mimo-V2.5 和 Qwen3.6 各种规模的模型。就我们所知，JIT-Agent 是首个专为即时支架生成而构建的模型；它确立了支架智能这一与模型扩展正交、可训练、可迁移且能持续积累的智能体能力维度。

项目主页

<https://bingreeky.github.io/JIT-site>

GitHub

<https://github.com/bingreeky/JIT>

Hugging Face

<https://huggingface.co/JIT-Agent>

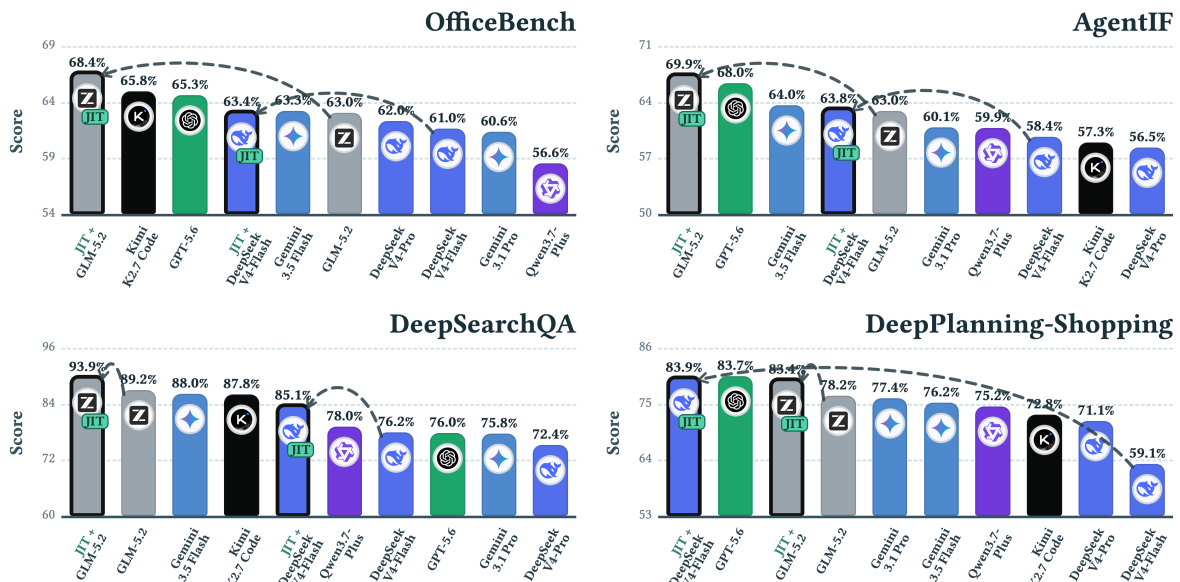


图 1: 四个代表性智能体基准上的排行榜。JIT-Agent 生成的支架在深度研究、日常工作、规划和工作区任务上持续提升底层骨干智能体，表明即时支架合成带来的收益超出了单纯扩展模型的效果。

1 引言

智能体支架至关重要。大语言模型智能体的能力由两个紧密耦合的因素共同决定：产生推理与动作的**基础模型**，以及将该模型置于闭环执行环境中的**智能体支架** (Meng et al., 2026b; Zhou et al.,

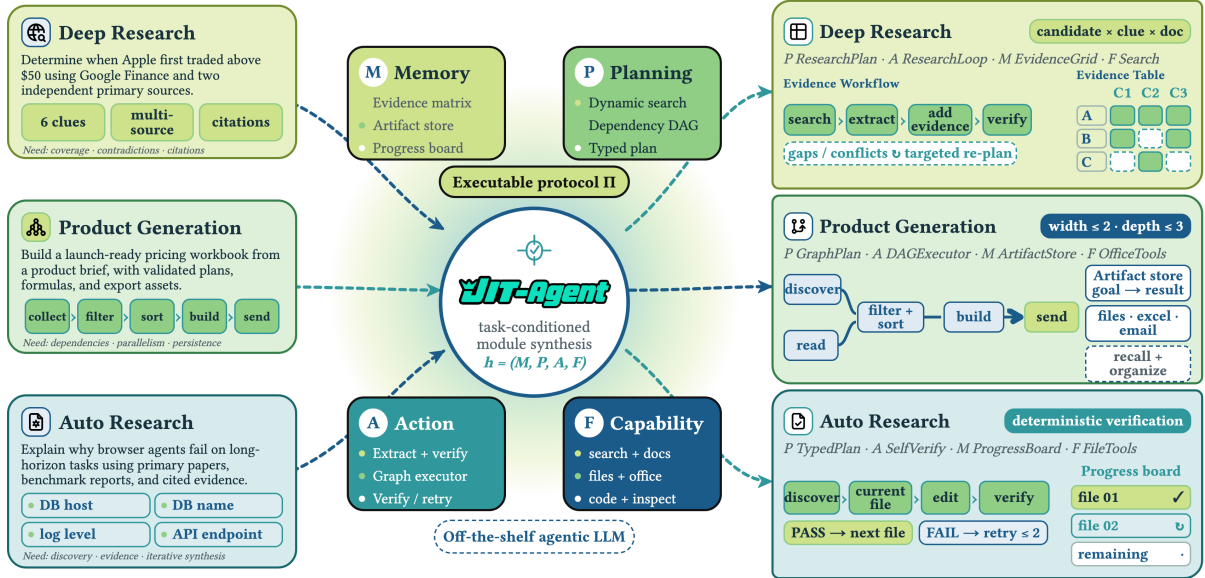


图 2: Jit-Agent 概览。给定一项任务，Jit-Agent 通过实例化（而非简单拼接）记忆、规划、动作和能力四个模块，构造面向该问题的智能体支架。因此，不同的任务结构会诱导出不同的可执行协议与状态组织方式；图中分别展示了面向深度研究、产品生成和自主研究的专用支架。

2026; Ning et al., 2026)。支架决定保留哪些历史、如何形成局部意图、暴露哪些工具或技能、如何执行动作，以及何时触发验证或恢复 (Meng et al., 2026b; Zhou et al., 2026)。强模型若置于错误的记忆、规划器或动作协议之后也可能失败；反过来，强支架也只有在模型能够理解并遵循它时才能释放能力 (Lee et al., 2026c; Lin et al., 2026)。因此，智能体的智能并非模型权重独有的属性，而是模型-支架组合的属性 (Lee et al., 2026c; Zhou et al., 2026)。由此，支架设计不是实现细节，而是决定智能体性能的一阶因素 (Meng et al., 2026b; Lin et al., 2026)。

预先 (AOT) 支架。上述观察催生了越来越多关于测试时支架优化的研究 (Lee et al., 2026c; Lin et al., 2026)。近期系统从轨迹和反馈中优化支架代码、提示、工具、记忆、技能或控制策略，在编码、终端使用、Web 交互和通用智能体领域取得了显著改进 (Lee et al., 2026c; Lin et al., 2026)。尽管这些方法彼此不同，许多方法都共享一个预先 (Ahead-of-Time, AOT) 假设：将支架视为在经验流上持续优化的耐久工件，并期望所得工件能够泛化至未来的任务、领域或模型版本 (Lee et al., 2026c; Lin et al., 2026)。当部署分布稳定且同质时，这是一个强大的范式 (Lin et al., 2026; Zhang et al., 2025b)。然而，它仍要求优化环在看到每个未来问题的确切结构之前，预先编译出一个普遍有用的支架 (Lee et al., 2026c)。

即时 (JIT) 支架。我们转而采用一种**即时 (Just-in-Time, JIT)**的支架构建观。不同任务显然需要不同的支架先验。广域搜索任务可能受益于并行证据探索 (Qin et al., 2025)；终端任务通常更适合精简、串行的 ReAct 环 (Yao et al., 2023a; Merrill et al., 2026)；深度研究任务需要围绕检索证据维护工作记忆 (Hu et al., 2025)；而 NL2Repo 风格的编码任务则天然由文件系统居中协调，文件系统存放补丁、测试、轨迹和仓库状态 (Yang et al., 2024; Ning et al., 2026)。简言之，合适的支架不仅取决于领域，还取决于具体实例 (Meng et al., 2026b; Zhou et al., 2026)。面对这些异质需求，优化单个 AOT 支架十分繁琐：系统必须搜索庞大的设计空间、积累足够多的轨迹，然后期待所得脚手架恰好适配下一项任务 (Lee et al., 2026c; Lin et al., 2026)。我们由此追问另一种可能性是否可行：**模型即支架 (Model-as-a-Harness)**，即由训练过的元智能体现场生成任务专用支架，再由任意智能体大语言模型在该支架下执行。

JIT-Agent. 我们提出 JIT-Agent，一种用于即时支架生成与演化的紧凑元智能体。¹ 在推理时，JIT-Agent 接收任务规范、协议、可执行工具/技能注册表，以及检索得到的一小段既往支架上下文，并输出为手头任务量身定制的可执行支架。所得支架包装一个现成的智能体大语言模型并负责执行。随着反馈与轨迹不断积累，JIT-Agent 可修订支架并更新支架档案，从而在生成器本身保持固定的情况下实现测试时支架演化。训练这样的系统面临三项挑战：① **适应性**，即如何让生成的支架匹配任务；② **可靠性**，即如何保证行为可执行，并在合成失败时恢复 (Pan et al., 2026a; Ning et al., 2026)；以及 ③ **可演化性**，即如何将执行反馈转化为未来更强的支架 (Lee et al., 2026c; Lin et al., 2026)。这三项要求共同定义了我们所说的**支架智能**：

支架智能

支架智能是构建并完善模型赖以行动的运行脚手架的能力。它有三个决定性属性：**适应性**，使支架匹配任务与骨干模型；**可靠性**，产生可执行行为，并在合成失败时恢复；**可演化性**，将执行反馈转化为未来更强的支架。固定支架内的能力抵达边界之处，正是支架智能开始之处。

JIT-Agent 在协议诱导的支架空间上采用三阶段训练方案来实现支架智能 (Meng et al., 2026b; Zhou et al., 2026)。首先，我们将智能体支架分解为由**记忆**、**规划**、**动作**和**能力编排**模块组成的四元组，并构建 HARNESSFACTORY：在这一公共接口下汇集代表性脚手架的统一代码库 (Hu et al., 2025; Erdogan et al., 2025a; Yao et al., 2023a; Xu and Yan, 2026)。遵循“代码承载支架”的观点 (Ning et al., 2026; Pan et al., 2026a)，这一表示使支架生成成为可能：JIT-Agent 生成结构化的可执行模块，而不是不受约束的智能体程序。随后，① **阶段 I** 从教师生成的协议合规样例中学习任务条件化定制，使生成器在共享设计空间中具备适应性。其次，② **阶段 II** 将失败的生成结果转化为有界修复轨迹，把编译器错误、接口不匹配和运行时故障变成可靠恢复的监督信号。第三，③ **阶段 III** 引入**演化式组解耦策略优化** (Evolutionary Group-Decoupled Policy Optimization, Evo-GDPO)，分别归一化奖励、延迟和成本，同时优化 JIT-Agent，使其提出能够超越当前档案前沿的支架。这一步把测试时支架改进从外部搜索启发式转变为训练所得能力，从而提供可演化性。通过这条流水线，我们训练得到 **JIT-Agent-27B**，一种能够定制、修复和演化任务专用支架的**支架智能模型**，将支架工程从 AOT 工件转向原生 JIT 合成。

实验发现。 结果表明，支架智能既能提升高效骨干模型，也能进一步拓展本已强大的模型。配备 JIT-Agent 的 DeepSeek-V4-Flash 在 DeepSearchQA、PinchBench 和 OdysseyBench 上分别超过 GPT-5.6 9.1、8.7 和 4.3 分。对于本已强大的 GLM-5.2，JIT-Agent 进一步抬高上限，在 xBench-DS 和 AgentIF 上分别取得 +12.0 和 +6.9 的显著绝对增益。除这些醒目的结果外，JIT-Agent 生成的支架在性能上可与 OpenCode、Claude Code 等成熟智能体运行时竞争，并能在 DeepSeek V4、Mimo-V2.5 和 Qwen3.6 不同规模的变体上持续带来收益。

贡献。 我们的贡献概括如下：

- **支架智能。** 我们将**模型即支架**形式化为一种新的智能体改进范式，并把**支架智能**定义为通过学习获得的、构建、修复和演化智能体运行脚手架的能力。
- **实用方案。** 我们提出 JIT-Agent 及其三阶段训练流水线，结合四模块可执行支架协议、HARNESSFACTORY、定制学习、反馈驱动修复，以及用于测试时支架演化的 Evo-GDPO。
- **实验验证。** 我们在异质智能体基准和模型骨干上验证 JIT-Agent，结果表明 JIT 生成的支架能够改善原生智能体、与强大的固定支架竞争，并推动骨干-支架组合迈向更优的成本-性能前沿。

2 相关工作

支架工程与模块化。 支架工程仍呈现出显著的多样性，覆盖 Claude Code、Codex、Hermes 和 OpenClaw 等编码与通用运行时 (Anthropics, 2025; OpenAI, 2025; Nous Research, 2026; OpenClaw Contributors, 2026)、具身系统 (Zhang et al., 2026e; Lee et al., 2026b)、搜索智能体 (Apodex Team,

¹JIT-Agent 延续了我们推动智能体系统递归设计与自演化的整体工作。在将支架作为整体处理之前，我们已研究单个支架组件层面的模块化设计与演化：MEMEVOLVE 中的记忆 (Zhang et al., 2025b) 和 TODOEVOLVE 中的规划 (Liu et al., 2026a)。JIT-Agent 是这条研究路线的统一延伸，将组件级自设计拓展到完整的运行支架。

表 1: 支架优化范式。 构建方式区分预先搜索得到的支架、随后由测试时反馈编辑的预先支架, 以及为每项任务即时生成的支架。实例合成、支架模型、学习式修复和在线演化分别表示一种方法是否直接合成实例专用支架、训练生成器、从失败执行轨迹中学习修复, 以及在部署后持续改进。

方法	构建方式	实例合成	支架模型	学习式修复	在线演化
AutoHarness (Lou et al., 2026)	AOT (搜索)	✗	✗	✗	✗
Meta-Harness (Lee et al., 2026c)	AOT (搜索)	✗	✗	✗	✗
AHE (Lin et al., 2026)	AOT (搜索)	✗	✗	✗	✗
Adaptive AH (Liu et al., 2026c)	AOT (测试时编辑)	✗	✗	✗	✓
TTHE (Nie et al., 2026b)	AOT (测试时编辑)	✗	✗	✗	✓
RHI (Lee et al., 2026a)	AOT (测试时编辑)	✗	✗	✗	✓
Harness-R1 (Shao et al., 2026)	AOT (测试时编辑)	✗	✓	✓	✓
JIT-Agent (本文)	JIT	✓	✓	✓	✓

2026; Qin et al., 2025), 以及 Mem0、EverMemOS 和 MemEvolve 等以记忆为中心的支架 (Chhikara et al., 2025; Hu et al., 2026a; Zhang et al., 2025b)。在这些差异之下, 支架是决定智能体观察、记忆、规划、调用、验证和执行内容的运行层, 因此能力是模型-支架组合而非模型权重本身的属性 (Meng et al., 2026b; Zhou et al., 2026; Pan et al., 2026a; Lee et al., 2026c)。反复出现的设计选择包括: (i) **记忆**, 用于整理工作上下文并保存跨会话经验 (Hu et al., 2026b, 2024a; Yan et al., 2025; Zhang et al., 2025c); (ii) **规划**, 形成并修订子目标, 以更新智能体的行动方向 (Cao et al., 2025; Erdogan et al., 2025b; Jin et al., 2025; Qin et al., 2025); (iii) **控制环与目标管理**, 执行目标、监控进度、反思结果并从失败中恢复 (Yao et al., 2023a; Shinn et al., 2023; Madaan et al., 2023; Sun et al., 2023); (iv) **工具分配**, 从大型 API 空间中选择可调用资源并确定其顺序 (Qin et al., 2023; Yang et al., 2026; Qian et al., 2025; Wang et al., 2025a); 以及 (v) **技能编排**, 从不断增长的技能库中发现、检索、组合和委派可复用过程 (Xu and Yan, 2026; Zheng et al., 2025; Li et al., 2026a; Zhou et al., 2026)。因此, 模块化已成为该领域共同的设计语言: 表示方式可以不同, 但显式接口使支架组件能够独立替换与优化 (Chen et al., 2026b; Ning et al., 2026; Pan et al., 2026a; Lin et al., 2026; Lee et al., 2026c)。例如, HarnessX 将运行时分解为提示、工具、记忆和控制流, 而 Code-as-Agent-Harness 则将其组织为接口、机制和多智能体扩展层 (Chen et al., 2026b; Ning et al., 2026)。因此, 我们采用紧凑分解 $h = (M, P, A, F)$, 四个模块分别构建记忆视图、形成下一条指令、推进动作环并编排工具与技能; 它提供最小可执行契约, 同时保留每个模块内部的可扩展性 (Hu et al., 2026b; Cao et al., 2025; Yao et al., 2023a; Xu and Yan, 2026)。

支架优化。 完成模块化后, 支架构建便从固定的工程选择转变为针对可执行工件的优化问题 (Khattab et al., 2023; Yuksekgonul et al., 2024; Hu et al., 2024b; Shang et al., 2024)。优化范围已从提示与声明式语言模型流水线扩展至工作流图、角色分配和完整支架代码, 使反馈既能修改局部指令, 也能修改全局控制结构 (Khattab et al., 2023; Zhang et al., 2024; Hu et al., 2024b; Shang et al., 2024; Lee et al., 2026c)。**闭环演化。** 在支架层面, 近期方法大多实现“诊断-提议-验证”循环: 执行轨迹暴露行为弱点, 候选编辑修改提示、记忆、工具、中间件或控制流, 而验证或偏好信号决定哪些更改得以保留 (Lee et al., 2026c; Lin et al., 2026; Zhang et al., 2026b; Pan et al., 2026b; Chen et al., 2026b)。近期工作进一步将这一循环扩展到递归式、层次式和协作式自我改进, 同时约束逐组件演化以改善迁移 (Lee et al., 2026a; Zhou, 2026; Nie et al., 2026a; Zhang et al., 2026c)。其监督形式从文本梯度和以轨迹为依据的可观测性, 到回归测试、回溯式自偏好和有类型原语替换不等, 但共同产物都是持续改进的可执行脚手架 (Yuksekgonul et al., 2024; Lin et al., 2026; Zhang et al., 2026b; Pan et al., 2026b; Chen et al., 2026b)。**AOT 与 JIT。** 尽管这些支架可能具备迁移性, 这些方法主要仍依据累积的经验分布优化一个耐久工件; 相对于新实例的确切结构而言, 它们属于预先方法 (Zhang et al., 2024, 2025a; Yuan et al., 2024; Chen et al., 2023)。我们则将支架搜索摊销到 JIT-Agent 中: 生成器在推理时合成任务条件化支架, 并可依据执行反馈进一步修订它, 使优化从重复的工件搜索转向学习得到的即时构建 (Meng et al., 2026b; Zhou et al., 2026; Ning et al., 2026; Pan et al., 2026a; Xu and Yan, 2026)。表 1 明确展示了这一区别。

3 统一支架代码库

3.1 模块化支架设计空间

智能体支架是将基础模型转化为闭环智能体的运行层 (Meng et al., 2026a; Zhou et al., 2026): 它决定保留既往交互中的哪些内容 (Hu et al., 2026b)、如何形成中间指令 (Cao et al., 2025)、每个阶段暴露哪些外部能力 (Xu and Yan, 2026; Yang et al., 2026), 以及如何推进控制。因此, 支架生成定义在程序而非不受约束的文本之上。我们区分原始生成空间 $\mathcal{G}$ 、语法有效的支架 $\mathcal{H}^{\text{syn}}$ 、符合协议的支架 $\mathcal{H}_{\Pi}$ 及其中的可执行子集 $\mathcal{H}_{\Pi}^{\text{exec}}$, 它们满足 $\mathcal{G} \supseteq \mathcal{H}^{\text{syn}} \supseteq \mathcal{H}_{\Pi} \supseteq \mathcal{H}_{\Pi}^{\text{exec}}$ 。固定协议 Π 规定模块模式与接口、模块生命周期、验证规则和共享执行语义。它消除了语言和运行时中的偶然差异, 同时保留区分现有支架的主要运行选择 (Wu et al., 2023; Microsoft, 2024; LangGraph, 2025; Luo et al., 2025)。

支架可组合性

可组合性已成为现代支架设计中的一等关切。例如, DeepSeek Harness 采用“万物皆插件”(Everything is a Plugin) 的架构, 将运行时表示为通过显式依赖关系连接的模块 (Shi et al., 2026)。可组合性对于 JIT 生成同样关键: 它把支架构建从不受约束的程序合成转化为在有类型、可重组设计空间中的装配。

JIT-Agent 通过四个可互操作模块显式表达这些选择: 如何压缩历史、如何形成局部意图、如何编排工具与技能, 以及如何推进控制。令 τ 为一项任务, π_{ψ} 为冻结的骨干执行器, C_{τ} 为该任务可用的能力注册表 (例如工具、API 和技能)。使用 π_{ψ} 运行支架 h 会产生闭环轨迹

$$\xi \sim \text{Rollout}(\tau, \pi_{\psi}, h, C_{\tau}; \Pi) = (s_1, e_1, o_1, \dots, s_T, e_T, o_T), \quad (1)$$

其中, $s_t \in \mathcal{S}$ 是所维护的控制器状态, e_t 是发出的工具调用或终止输出, $o_t \in \mathcal{O}$ 是所得观察, 而 T 是由协议或预算限制的停止时刻。我们的关键假设是, 每个 $h \in \mathcal{H}_{\Pi}$ 都允许如下模块化分解:

$$h = (M, P, A, F) \in \mathfrak{M} \times \mathfrak{P} \times \mathfrak{A} \times \mathfrak{F}, \quad (2)$$

其中, M 、 P 、 A 和 F 分别表示记忆、规划、动作和能力编排模块, 而 $\mathfrak{M}$ 、 $\mathfrak{P}$ 、 $\mathfrak{A}$ 和 $\mathfrak{F}$ 是各自与协议兼容的实现空间。该元组遵循上述概念分解; 在运行时, 其依赖顺序为 $M \rightarrow P \rightarrow F \rightarrow A$ 。所有模块都面向同一个冻结的 π_{ψ} 运行, 下文省略这种共同的运行时依赖。

定位

Codex、Claude Code 和 DeepSeek Harness 等生产级支架所暴露的机制远比我们的四模块实例丰富 (Shi et al., 2026; Anthropic, 2025; OpenAI, 2025)。这一差距是有意为之。本工作并非要求模型复现完整的生产运行时, 而是要确立一个更基础的结果: 即便支架十分紧凑, 只要即时生成, 也能带来显著收益。此外, 我们认为 JIT 范式与生产支架工程互为补充。随着模型越来越多地参与自身运行时的设计与修订, 支架智能的训练方案在更大规模上也会同样重要。我们将当前设计空间视为一个起点。

该协议同时维护不可变事件历史 $\xi_{<t}$ 和可变控制器状态 s_t , 二者按如下方式交互:

$$v_t = M(\xi_{<t}, s_t) \in \mathcal{V}, \quad \text{历史} \rightarrow \text{视图}, \quad (3)$$

$$d_t = P(\tau, s_t, v_t) \in \mathcal{D}_{\text{dir}}, \quad \text{视图} \rightarrow \text{局部指令}, \quad (4)$$

$$C_t = F(C_{\tau}, s_t, v_t, d_t) \subseteq C_{\tau}, \quad \text{以指令为条件的能力编排}, \quad (5)$$

$$(s_{t+1}, e_t) = A(s_t, \tau, v_t, d_t, C_t) \in \mathcal{S} \times \mathcal{A}, \quad \text{控制更新与动作发出}, \quad (6)$$

其中, $\mathcal{V}$ 和 $\mathcal{D}_{\text{dir}}$ 分别是视图空间和指令空间, $\mathcal{A} = \mathcal{U} \sqcup \mathcal{Y}$ 是可执行调用与终止输出的不交并。注册表 C_{τ} 包含可调用工具 (例如 bash 工具、API 和 MCP (Sarkar and Sarkar, 2025)) 以及更高层的智能体技能 (Xu and Yan, 2026; ClaudeCode, 2025)。没有显式规划器的支架仍可通过空指令 $d_{\emptyset} \in \mathcal{D}_{\text{dir}}$ 保持类型一致; $P_{\emptyset}$ 会在每一步返回该空指令。换言之, 记忆从已实现历史中构建视图, 规划将该

表 2: 种子库 $\mathcal{B}_0$: 13 个手工编写的支架, 均实例化四模块协议 Π 。每一行都是一个完整支架; 各列依次对应记忆、规划、动作和能力编排。

支架	♣ 记忆 $M \in \mathfrak{M}$	♦ 规划 $P \in \mathfrak{P}$	♥ 动作 $A \in \mathfrak{A}$	♠ 能力编排 $F \in \mathfrak{F}$
ReAct [83]	完整历史	无显式规划器	ReAct	完整注册表
PLAN-AND-EXECUTE [16]	完整历史	线性路线图	ReAct	完整注册表
ReSum [77]	ReSum 记忆	无显式规划器	ReAct	完整注册表
FLASH-SEARCHER [60]	完整历史	DAG 规划	ReAct	完整注册表
GAM [79]	GAM 检索	DAG	ReAct	完整注册表
MEMOBRAIN [59]	推理图	无显式规划器	标记引导执行	完整注册表
AGGAGENT [32]	隔离的回滚历史	无显式规划器	多回滚聚合	完整注册表
OAGENT [100]	协调器历史	无显式规划器	集成投票	完整注册表
AGENTFOLD [84]	AgentFold 记忆	DAG	ReActFold	完整注册表
HiAGENT [22]	层次记忆	无显式规划器	ReAct	完整注册表
DEEPAGENT [34]	三层记忆	无显式规划器	标记引导执行	工具搜索
ROMA [1]	上下文隔离	原子化器 + DAG	递归执行	完整注册表
AORCHESTRA [63]	上下文隔离	原子化器 + DAG	递归执行	智能体委派

视图转化为局部指令, 能力编排激活相关外部工具或智能体技能, 而动作模块使用组装后的上下文, 同时更新控制器状态并发出下一动作。内核随后按如下方式解释发出的动作:

$$O_t = \begin{cases} \text{Exec}(e_t; C_t), & e_t \in \mathcal{U}, \\ \perp, & e_t \in \mathcal{Y}, \end{cases} \quad \xi_{\leq t} = \xi_{< t} \oplus (s_t, e_t, O_t), \quad (7)$$

其中, $\text{Exec} : \mathcal{U} \times 2^{C_t} \rightarrow \mathcal{O}$ 是共享执行内核, $\perp \in \mathcal{O}$ 是终止空观察, $\oplus$ 表示向轨迹追加一个事件。执行从 $\xi_{< 1} = \emptyset$ 和协议定义的 $s_1 \in \mathcal{S}$ 开始, 在 $e_t \in \mathcal{Y}$ 时终止, 并产生 $y = e_t \in \mathcal{Y}$ 。这一分解把原本异质的程序转化为 $\mathfrak{M} \times \mathfrak{P} \times \mathfrak{A} \times \mathfrak{F}$ 中可比较的坐标:

- **标准 ReAct** 可写作 $h_{\text{ReAct}} = (M_{\text{full}}, P_{\emptyset}, A_{\text{react}}, F_{\text{all}})$, 其中 A_{react} 是标准 ReAct 环, $P_{\emptyset}$ 表示没有显式规划器, M_{full} 不做上下文管理而保留运行历史, F_{all} 则暴露完整的工具/技能注册表。
- Codex (OpenAI, 2025) 和 OpenCode (OpenCode-AI, 2025) 等**工程化 ReAct 变体**仍符合相同的脚手架: $(M_{\text{compact}}, P_{\text{todo}}, A_{\text{react}}, F_{\text{all}})$ 。其中动作内核仍为 ReAct 风格, 但 M_{compact} 会在接近上下文上限时压缩历史, 而 P_{todo} 维护显式的任务待办列表。
- **递归智能体**。ROMA (Alzu'bi et al., 2026)、AOrchestra (Ruan et al., 2026) 和递归语言模型 (RLM) (Zhang et al., 2026a) 等递归架构同样可以由 $h_{\text{rec}} = (M_{\text{subproblem}}, P_{\text{decomp}}, A_{\text{rec}}, F_{\text{route}})$ 之类的选择刻画。在此类系统中, 主编排器可通过 A_{rec} 生成子智能体, 在 P_{decomp} 中维护待办式分解, 在 $M_{\text{subproblem}}$ 中隔离智能体上下文, 并通过 F_{route} 向每个子智能体分配工具或技能。

在这一模块化设计空间的基础上, 接下来我们介绍 HARNESSTORY。它用于检验 $\mathcal{H}_{\Pi}$ 的表达能力, 并为 JTT-Agent 的元支架设计提供多样化素材。

3.2 HarnessFactory

在共享协议 Π 和公共内核下, HARNESSTORY 重新实现了 13 个有代表性的当代智能体脚手架: ReAct (Yao et al., 2023b)、PLAN-AND-EXECUTE (Erdogan et al., 2025b)、ReSum (Wu et al., 2026)、FLASH-SEARCHER (Qin et al., 2025)、GENERAL AGENTIC MEMORY (GAM) (Yan et al., 2025)、MEMOBRAIN (Qian et al., 2026)、AGGAGENT (Lee et al., 2026d)、OAGENT (Zhu et al., 2025)、AGENT-FOLD (Ye et al., 2025)、HiAGENT (Hu et al., 2024a)、DEEPAGENT (Li et al., 2026b)、ROMA (Alzu'bi et al., 2026) 和 AORCHESTRA (Ruan et al., 2026)。它们提供异质的记忆、规划、动作与能力编排策略。因此, 种子库 $\mathcal{B}_0$ 包含 $K_0 = 13$ 个与协议兼容的支架。表 2 给出了完整清单。

该库具有双重作用: 从 $\mathcal{B}_0$ 采样的支架为阶段 I 合成提供锚点, 而后续档案状态则提供用于评估新设

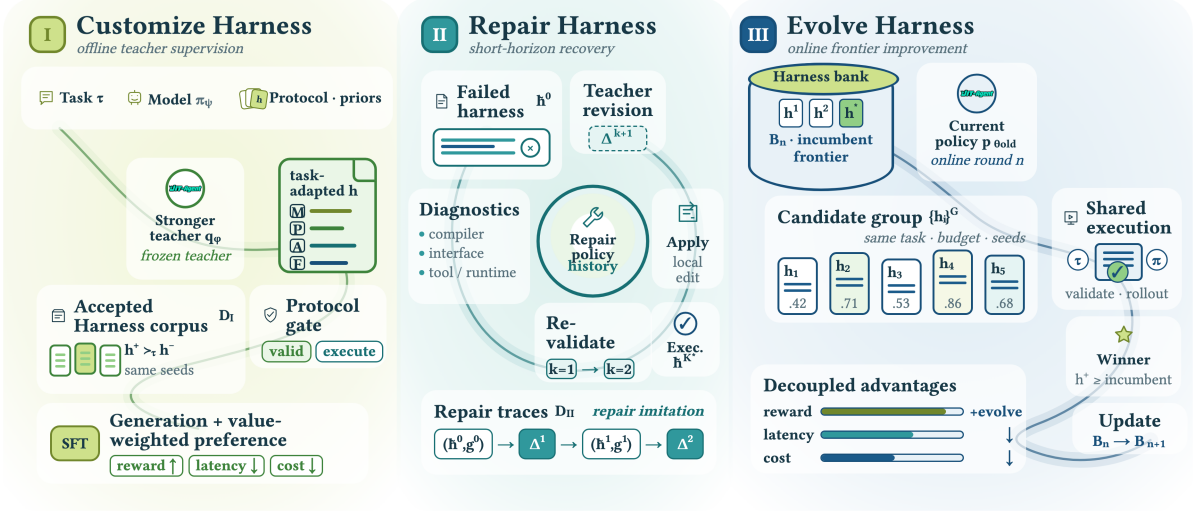


图 3: JIT-Agent 的训练流水线。阶段 I 学习任务条件化的支架定制。阶段 II 将失败支架和执行诊断转化为有界修复轨迹。阶段 III 将候选支架与当前库进行比较，优化解耦的奖励、延迟和成本优势，并保留改善前沿的设计，从而执行在线演化。

计的先验种群。随着系统运行， $\mathcal{B}_0$ 增长为 $\mathcal{B}_n \supseteq \mathcal{B}_0$ ；其中每个条目都把保留的支架与其任务及观测到的奖励、延迟和成本关联起来。第 4.3 节对这一演化过程给出形式化描述。

4 训练流水线

训练遵循 JIT-Agent 在推理时的生命周期：首先合成任务条件化支架，然后恢复不稳定的生成结果，最后学习如何在线利用更强的档案状态继续改进。令 $\tau \sim \mathcal{D}_{\text{task}}$ 表示从训练分布中抽取的任务， C_τ 表示其能力注册表， $\mathcal{E}_\tau$ 表示来自支架库的一小段参考上下文。我们将生成上下文定义为 $c_\tau = (\tau, \Pi, C_\tau, \mathcal{E}_\tau)$ ，并从 $p_\theta(h | c_\tau)$ 采样支架。随后，冻结的 executor $\pi_\psi \sim \mathcal{M}$ 运行每个支架，以完成验证和效用度量。共同目标为

$$\theta^* = \arg \max_{\theta} \mathbb{E}_{\tau \sim \mathcal{D}_{\text{task}}, \pi_\psi \sim \mathcal{M}, h \sim p_\theta(\cdot | c_\tau)} [U(\tau, \pi_\psi, h)], \quad (8)$$

其中， U 使用任务奖励、延迟和货币成本评估模型-支架组合所诱导的轨迹。由于 p_θ 在 $\mathcal{G}$ 中生成，并不通过构造天然满足可执行性，因此每个输出都要接受协议验证器检查。我们用 $\text{Valid}_\Pi(h; \tau, \pi_\psi, C_\tau) \in \{0, 1\}$ 表示其有效性标志；检查失败时还会返回结构化诊断报告。阶段 I 的离线监督、阶段 II 的修复轨迹学习和阶段 III 的在线策略改进共同实例化了式 (8)。

4.1 阶段 I: 定制支架

数据准备。 阶段 I 使用冻结且更强的教师 q_ϕ ，在固定四模块协议下合成适应任务的支架。对于每项任务，从种子库中与任务类型匹配的子集 $\mathcal{B}_0^{(d(\tau))}$ 采样三个参考脚手架：

$$\mathcal{E}_\tau = \{h^{(1)}, h^{(2)}, h^{(3)}\} \sim \text{Sample}_3(\mathcal{B}_0^{(d(\tau))}), \quad h^{\text{teach}} \sim q_\phi(\cdot | c_\tau), \quad (9)$$

其中 $d(\tau)$ 表示任务类型。教师接收任务、协议、能力注册表和采样得到的脚手架。只有通过协议验证与执行检查的生成结果才会保留，从而得到

$$\mathcal{D}_1 = \left\{ (\tau, \pi_\psi, C_\tau, \mathcal{E}_\tau, h^{\text{teach}}) \mid \text{Valid}_\Pi(h^{\text{teach}}; \tau, \pi_\psi, C_\tau) = 1 \right\}. \quad (10)$$

阶段 I 的训练任务来自不同来源，包括文献 (Wu et al., 2025; Shi et al., 2025; Zhang et al., 2026d; Bai et al., 2026; Burgess et al., 2026; Song et al., 2026) 所用任务，以及一部分合成任务。

训练设置。阶段 I 使用两个耦合目标。第一个目标是在被接受的教师生成结果上进行标准监督微调：

$$\mathcal{L}_I^{\text{gen}}(\theta) = -\mathbb{E}_{(\tau, \pi_\psi, C_\tau, \mathcal{E}_\tau, h^{\text{teach}}) \sim \mathcal{D}_I} \sum_{j=1}^{|h^{\text{teach}}|} \log p_\theta(y_j^{\text{teach}} | y_{<j}^{\text{teach}}, c_\tau), \quad (11)$$

其中 y_j^{teach} 是第 j 个目标词元。该目标教会 JIT-Agent 把任务和一小段参考上下文直接映射为符合协议的支架。仅符合协议还不够：可执行支架在任务奖励、延迟和货币成本上仍可能相差很大。因此，我们在相同骨干模型和评测种子下比较候选项；只有当奖励提升、两个效率维度均未退化，且至少一个效率维度严格改善时，才保留该偏好。

$$h^+ \succ_\tau h^- \iff r^+ > r^- \wedge \ell^+ \leq \ell^- \wedge \kappa^+ \leq \kappa^- \wedge (\ell^+ < \ell^- \vee \kappa^+ < \kappa^-), \quad (12)$$

$$\Delta_{\text{val}}(\tau; h^+, h^-) = \alpha_r(r^+ - r^-) + \alpha_\ell[\ell^- - \ell^+]_+ + \alpha_\kappa[\kappa^- - \kappa^+]_+,$$

其中， $(r^\pm, \ell^\pm, \kappa^\pm)$ 是 (h^+, h^-) 多次回滚的平均值， $[x]_+ = \max(x, 0)$ ，而 $\alpha_r, \alpha_\ell, \alpha_\kappa \geq 0$ 控制三个价值差。令 $\mathcal{D}_I^{\text{pref}}$ 收集由此得到的偏好元组。我们优化以下以参考模型为锚的目标：

$$\mathcal{L}_I^{\text{pref}}(\theta) = -\mathbb{E}_{(\tau, \pi_\psi, C_\tau, \mathcal{E}_\tau, h^+, h^-) \sim \mathcal{D}_I^{\text{pref}}} \left[\Delta_{\text{val}} \cdot \log \sigma \left(\beta_{\text{pref}} \log \frac{p_\theta(h^+ | c_\tau)}{p_\theta(h^- | c_\tau)} - \beta_{\text{pref}} \log \frac{p_{\text{ref}}(h^+ | c_\tau)}{p_{\text{ref}}(h^- | c_\tau)} \right) \right]. \quad (13)$$

这里， σ 是 logistic sigmoid， $\beta_{\text{pref}} > 0$ 控制偏好的锐度， p_{ref} 是冻结的阶段 I SFT 检查点；在对数比值中， $\log p(h | \cdot)$ 表示按长度归一化的序列对数似然。完整的阶段 I 目标为 $\mathcal{L}_I(\theta) = \mathcal{L}_I^{\text{gen}}(\theta) + \lambda_{\text{pref}} \mathcal{L}_I^{\text{pref}}(\theta)$ ，其中 $\lambda_{\text{pref}} \geq 0$ 。生成模仿建立协议有效的结构，而偏好学习则使模型偏向同时更有效、更高效的支架。

4.2 阶段 II：修复支架

数据准备。阶段 I 针对协议合规性进行优化，但仍不能保证可执行性：有些生成的支架无法通过静态或运行时验证，因此永远不会进入 $\mathcal{D}_I$ 。阶段 II 不丢弃这些失败，而是将其转化为修复监督。令

$$\mathcal{D}_I^{\text{fail}} = \left\{ (\tau, \pi_\psi, C_\tau, \mathcal{E}_\tau, \tilde{h}^{(0)}, g^{(0)}) \mid \tilde{h}^{(0)} \notin \mathcal{H}_\Pi^{\text{exec}} \right\}, \quad (14)$$

其中 $\tilde{h}^{(0)}$ 是阶段 I 中失败的支架， $g^{(0)}$ 是对应的诊断报告，包括编译器错误、接口不匹配、工具调用失败和运行时异常。对于每次失败，教师从补丁空间 $\mathcal{P}$ 提出结构化修订 $\Delta^{(k+1)} \in \mathcal{P}$ ，再由 Apply 确定性地把它应用到当前支架：

$$\tilde{h}^{(k)}, g^{(k)} \longrightarrow \Delta^{(k+1)} \longrightarrow \tilde{h}^{(k+1)} = \text{Apply}(\tilde{h}^{(k)}, \Delta^{(k+1)}), \quad k \geq 0. \quad (15)$$

每次修订后，验证过程都会产生下一份报告 $g^{(k+1)}$ 。我们只保留在两轮修复内变得可执行的轨迹。定义

$$K^* = \min \left\{ k \in \{1, 2\} : \text{Valid}_\Pi(\tilde{h}^{(k)}; \tau, \pi_\psi, C_\tau) = 1 \right\}, \quad (16)$$

只有 K^* 定义良好时才构建阶段 II 监督。这使语料聚焦于现实中能够局部恢复的失败，而非需要彻底重新设计的失败。

训练设置。阶段 II 随后训练 JIT-Agent 模仿成功的修复转移，而非进行一次性合成。对于每条保留的轨迹，令

$$\mathcal{D}_{II} = \left\{ (\tau, \pi_\psi, C_\tau, \mathcal{E}_\tau, \mathcal{R}_{K^*}) \mid (\tau, \pi_\psi, C_\tau, \mathcal{E}_\tau, \tilde{h}^{(0)}, g^{(0)}) \in \mathcal{D}_I^{\text{fail}}, K^* \text{ exists} \right\}, \quad (17)$$

其中 $\mathcal{R}_{K^*} = \{(\tilde{h}^{(j)}, g^{(j)}, \Delta^{*(j+1)})\}_{j=0}^{K^*-1}$ ，每个 $\Delta^{*(j+1)}$ 都是最终能够变为可执行的修复轨迹上的教师修订。修复目标以完整历史为条件：

$$\mathcal{L}_{II}(\theta) = -\mathbb{E}_{\mathcal{D}_{II}} \sum_{k=0}^{K^*-1} \log p_\theta(\Delta^{*(k+1)} \mid c_\tau, \{(\tilde{h}^{(j)}, g^{(j)})\}_{j=0}^k). \quad (18)$$

由于 $K^* \leq 2$ ，所得监督恰好针对部署时至关重要的短视野修复情形：给定一个几乎正确但不稳定的支架，JIT-Agent 学习利用执行反馈生成少量高杠杆修订，以恢复符合协议的执行。

4.3 阶段 III：学习演化支架

阶段 III 将测试时支架演化本身视为一种可训练能力。其目标不只是恢复或模仿先前观测到的支架设计，而是优化 JIT-Agent，使其在测试时能够反复提出超越既往设计的支架，成为未来更强的参考，并持续推进支架前沿。受文献 (Liu et al., 2026b) 启发，我们将所得在线目标称为**演化式组解耦策略优化 (Evo-GDPO)**。

数据准备。阶段 III 的训练样例由三个同步来源构成：一个任务实例、一小组既往高质量支架设计，以及针对同一任务在线收集的新鲜执行反馈。具体而言，在第 n 个在线轮次，我们采样任务 τ ，从当前支架库 $\mathcal{B}_n$ 中检索小型参考集 $\mathcal{E}_{\tau,n}$ ，并构造 $c_{\tau,n} = (\tau, \Pi, C_\tau, \mathcal{E}_{\tau,n})$ 。模型提出候选项；它们与这些既往设计在相同的冻结执行器 $\pi_\psi \sim \mathcal{M}$ 、预算和评测种子下并列执行。在 $\mathcal{E}_{\tau,n}$ 中，奖励最高的支架成为唯一的现任支架；奖励相同时先取延迟更低者，再取成本更低者。其统计量记作 (b_r, b_ℓ, b_κ) 。

训练设置。给定这一上下文，Evo-GDPO 首先从当前策略中采样一组候选支架：

$$h_i \stackrel{\text{i.i.d.}}{\sim} p_{\theta_{\text{old}}}(\cdot | c_{\tau,n}), \quad i = 1, \dots, G, \quad (19)$$

其中 $G > 1$ 是组大小， θ_{old} 是回滚策略快照。每个候选项在执行前都要经过验证：若候选项经有界修复后仍无效，则获得最低任务奖励，但其修复延迟和成本仍计入效率度量。对于可执行候选项， r_i 、 $\bar{\ell}_i$ 和 $\bar{\kappa}_i$ 分别表示多次回滚中的奖励、平均延迟和平均货币成本。奖励通道居于首位；只有候选项保持现任支架的奖励时，效率通道才会激活。

$$\begin{aligned} R_i^{\text{rew}} &= r_i + \lambda_{\text{evo}} [r_i - b_r]_+, \\ R_i^{\text{lat}} &= \mathbb{I}[r_i \geq b_r] [b_\ell - \bar{\ell}_i]_+, \\ R_i^{\text{cost}} &= \mathbb{I}[r_i \geq b_r] [b_\kappa - \bar{\kappa}_i]_+. \end{aligned} \quad (20)$$

这里， $\lambda_{\text{evo}} \geq 0$ 控制奖励加成， $\mathbb{I}[\cdot]$ 为指示函数。Evo-GDPO 在合并前三个信号之前分别对其进行归一化，防止某个信号的数值尺度压倒其他信号。第二次批次级归一化用于稳定跨任务优化。

$$\begin{aligned} A_i^m &= \frac{R_i^m - \text{mean}(\{R_j^m\}_{j=1}^G)}{\text{std}(\{R_j^m\}_{j=1}^G) + \epsilon_{\text{num}}}, \quad m \in \{\text{rew}, \text{lat}, \text{cost}\}, \\ A_i^\Sigma &= w_{\text{rew}} A_i^{\text{rew}} + w_{\text{lat}} A_i^{\text{lat}} + w_{\text{cost}} A_i^{\text{cost}}, \quad w_{\text{rew}} > w_{\text{lat}} + w_{\text{cost}}, \\ \hat{A}_i^\Sigma &= \frac{A_i^\Sigma - \text{mean}_{\text{batch}}(A^\Sigma)}{\text{std}_{\text{batch}}(A^\Sigma) + \epsilon_{\text{num}}}. \end{aligned} \quad (21)$$

这些非负权重之和为一，而给定的不等式确保任务奖励占主导； $\epsilon_{\text{num}} > 0$ 是上述两个分母中的数值稳定项。最终策略更新采用由该聚合优势驱动的 PPO 风格裁剪目标。相较于标准 GRPO 风格训练，模型获得奖励不仅因为它在采样组内表现良好，更因为它超越了既往支架设计，并在保持奖励质量时以更高效率做到这一点。

$$\begin{aligned} \mathcal{L}_{\text{III}}^{\text{Evo-GDPO}}(\theta) &= -\mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|h_i|} \sum_{j=1}^{|h_i|} \min \left(\rho_{i,j}(\theta) \hat{A}_i^\Sigma, \text{clip}(\rho_{i,j}(\theta), 1 - \epsilon_{\text{clip}}, 1 + \epsilon_{\text{clip}}) \hat{A}_i^\Sigma \right) \right] \\ &\quad + \beta_{\text{KL}} \mathbb{E} [\text{KL}(p_\theta \| p_{\text{ref}})], \\ \rho_{i,j}(\theta) &= \frac{p_\theta(y_{i,j} | y_{i,<j}, c_{\tau,n})}{p_{\theta_{\text{old}}}(y_{i,j} | y_{i,<j}, c_{\tau,n})}. \end{aligned} \quad (22)$$

该期望遍及采样任务、冻结执行器，以及从 $p_{\theta_{\text{old}}}$ 得到的候选组； $|h_i|$ 是支架词元长度， $\epsilon_{\text{clip}} > 0$ 是裁剪半径， $\beta_{\text{KL}} \geq 0$ ，而 p_{ref} 是用于词元级 KL 惩罚的冻结阶段 II 检查点。优化后，我们保守地更新支架库：候选项必须达到或超过当前奖励前沿，并随后在奖励本身、延迟或成本三个前沿维度中的至少一个上严格改善，才会被保留。训练期间，该反馈同时更新 θ 与 $\mathcal{B}_n$ ；部署时， θ 保持冻结。

5 推理架构

JIT-Agent 支持两种推理模式：*静态推理*和*流式推理*；二者的区别在于任务结束后是丢弃经验，还是保留经验以支持后续任务。

静态推理。在这一模式下，我们引入一种轻量的测试时扩展形式：JIT-Agent 并行生成 N 个支架，从中选择一个，并且只执行所选支架。这样能够在不增加环境回滚次数的情况下提高候选多样性。所选支架遵循前述相同的验证与有界修复过程。

流式推理。流式推理模式旨在跨一系列任务向前传递有用经验。对于第 n 个任务 τ_n ，JIT-Agent 从当前库 $\mathcal{B}_n$ 检索信息，生成并选择支架 $h_n^\dagger$ ，然后执行一次。所得环境反馈仅用于判断该经验是否应更新支架库：

$$\begin{aligned}\xi_n &\sim \text{Rollout}(\tau_n, \pi_\psi, h_n^\dagger, C_{\tau_n}; \Pi), \\ m_n &= \text{Eval}(\xi_n), \\ \mathcal{B}_{n+1} &= \text{Update}_{\text{III}}(\mathcal{B}_n; \tau_n, h_n^\dagger, m_n), \\ \mathcal{E}_{\tau_{n+1}, n+1} &= \text{Retrieve}(\tau_{n+1}; \mathcal{B}_{n+1}),\end{aligned}\tag{23}$$

其中 $m_n = (r_n, \bar{\ell}_n, \bar{\kappa}_n)$ 包含奖励、平均延迟和平均货币成本。按照阶段 III 的保留规则，当执行完毕的支架没有带来可接受的改进时， $\text{Update}_{\text{III}}$ 保持 $\mathcal{B}_n$ 不变；否则，保留的支架会成为后续任务的潜在参考。因此，流式推理通过不断演化的支架库传递既往经验，而不会把环境反馈注入当前回滚，也不会更新模型参数。

6 实验与分析

6.1 实验设置

评测基准。我们在九个基准上评测 JIT-Agent，并将其分为四类任务：

- **深度研究：**BrowseComp-Plus (Chen et al., 2025b) (准确率)、DeepSearchQA (Gupta et al., 2026) (答案 F1) 和 xBench-DeepSearch (xBench-DS) (Chen et al., 2025a) (准确率)。
- **日常工作：**AgentIF-Oneday (Chen et al., 2026a) (归一化加权量表分数) 和 PinchBench (Pinch-Bench, 2026) (平均分)。
- **规划：**文献 (Zhang et al., 2026d) 中的 DeepPlanning-Shopping (购物车匹配率) 和 DeepPlanning-Travel (复合约束分数) 子集。
- **工作区：**OfficeBench (Wang et al., 2024) 和 OdysseyBench (Wang et al., 2025b) (两者均采用任务成功率)。

这些基准共同覆盖证据密集型搜索、长视野指令遵循、约束感知规划和多应用工作区执行。所有指标均按 0–100 尺度报告；越高越好。

骨干模型。JIT-Agent 基于 Qwen3.6-27B 训练。我们主要使用 GLM-5.2 (Z.ai, 2026) 和 DeepSeek-V4-Flash-Preview (DeepSeek-AI, 2026) 来实例化 JIT-Agent。这两个骨干模型提供互补的运行点：GLM-5.2 是强大的开放长视野模型，而 DeepSeek-V4-Flash 强调推理效率。为检验泛化能力，我们还使用了 Qwen3.6 和 MIMO-V2.5 系列中的其他模型。

基线。我们使用两组互补的基线。首先，广泛比较包括原生 Qwen3.7-Plus (Qwen Team, 2026)、GLM-5.2、DeepSeek-V4-Flash/Pro (两者均为预览版)、Kimi K2.7 Code (Moonshot AI, 2026)、GPT-5.6 (OpenAI, 2026) 和 Gemini 3.1 Pro/3.5 Flash (Google DeepMind, 2026a,b)。其次，在固定骨干模型的情况下，我们将 JIT-Agent 与五种可复用智能体支架进行比较：CLAUDE CODE (Anthropics, 2025)、CODEX (OpenAI, 2025)、OPENCODE (OpenCode-AI, 2025)、HERMES (Nous Research, 2026) 和 NANOBOT (HKUDS, 2026)。第一组衡量端到端竞争力，第二组则把支架设计的贡献与模型权重的贡献分离开来。

表 3: 九个智能体基准上的主要结果。以 JIT-Agent 开头的行采用与其原生对应项相同的骨干模型，但用 JIT 生成的支架替换默认脚手架。分数按 0–100 尺度报告；越高越好。每列排名第 1、第 2 和第 3 的结果分别以浅绿色、黄绿色和黄色标出，其中第 1 名还使用粗体。缺失评测以“—”表示。

模型	深度研究			日常工作		规划		工作区	
	BC+	DSQA	xBench	AgentIF	PinchBench	Shop	Travel	Office	Odyssey
Qwen3.7-Plus	70.5	78.0	75.0	59.9	80.5	75.2	52.1	56.6	67.7
GLM-5.2	72.0	89.2	76.0	63.0	87.0	78.2	62.8	63.0	75.3
DeepSeek-V4-Flash	68.1	76.2	70.1	58.4	81.7	59.1	54.8	61.0	71.0
DeepSeek-V4-Pro	71.4	72.4	79.0	56.5	61.1	71.1	55.2	62.0	72.0
Kimi K2.7 Code	73.6	87.8	79.0	57.3	76.1	72.8	56.9	65.8	69.9
GPT-5.6	76.9	76.0	81.0	68.0	84.2	83.7	84.9	65.3	68.7
Gemini 3.1 Pro	67.5	75.8	83.0	60.1	81.0	77.4	70.7	60.6	74.0
Gemini 3.5 Flash	75.0	88.0	85.0	64.0	74.2	76.2	50.3	63.3	78.0
JIT-Agent + GLM-5.2	78.0	93.9	88.0	69.9	93.3	83.4	83.0	68.4	78.7
JIT-Agent + DeepSeek-V4-Flash	74.0	85.1	82.0	63.8	92.9	83.9	61.3	63.4	73.0

6.2 主要结果

表 3 将 JIT 生成的支架与原生智能体骨干及前沿模型基线进行比较。我们报告三个深度研究基准、两个日常工作基准、两个规划基准和两个工作区基准。

同一骨干模型上的一致增益。在全部 18 个直接匹配的骨干–基准组合中，用 JIT 生成的支架替换默认脚手架都提升了性能。在 GLM-5.2 上，九个基准的平均分从 74.1 升至 81.8 (+7.7 分)；在 DeepSeek-V4-Flash 上，则从 66.7 升至 75.5 (+8.8 分)。最大增益出现在需要持续状态管理和约束跟踪的任务上：DeepSeek-V4-Flash 在 DeepPlanning-Shopping 上提高 24.8 分 (59.1 → 83.9)，GLM-5.2 在 DeepPlanning-Travel 上提高 20.2 分 (62.8 → 83.0)。搜索和开放式交互同样获益显著，例如 DeepSeek-V4-Flash 在 xBench-DS 和 DeepSearchQA 上分别提高 11.9 和 8.9 分。这些广泛增益表明，支架适配改变的不只是提示风格；它还改善了骨干模型分配上下文、分解长视野目标及协调外部动作的方式。

与前沿模型的竞争力。尽管采用开放骨干模型，配备 JIT 的系统仍在九个基准列中的八列取得最佳结果。尤其是，JIT-Agent + GLM-5.2 在七个基准上排名第一，在 DeepSearchQA、AgentIF 和 PinchBench 上分别达到 93.9、69.9 和 93.3；JIT-Agent + DeepSeek-V4-Flash 则以 83.9 领先 DeepPlanning-Shopping。后一组合在所有已报告基准上都超过了更强的 DeepSeek-V4-Pro 基线，平均优势为 8.7 分。DeepPlanning-Travel 是唯一未由配备 JIT 的模型领先的列；GLM-5.2 + JIT-Agent 在该列达到 83.0，仅比 GPT-5.6 低 1.9 分。总体而言，这些结果支持本文的核心主张：改进运行脚手架能够收回原本通过扩展骨干模型才能追求的大部分能力。

6.3 与先进智能体支架的比较

除与原生骨干模型比较外，我们还控制底层模型，只改变支架。我们在 DeepSeek-V4-Flash 和 Qwen3.6-Flash 上分别实例化每种支架，并在 DeepSearchQA、xBench-DS 和 AgentIF 上进行评测。除任务性能外，我们还报告每个样例的平均词元消耗与 API 成本，以区分有效编排带来的收益和单纯投入更多推理算力所得的收益。

控制骨干模型时的性能。JIT-Agent 在六个骨干–基准设置中的四个取得最高性能。在 DeepSeek-V4-Flash 上，它在 DeepSearchQA 上比最强固定支架高 4.7 分 (85.1 对 80.4)，在 xBench-DS 上高 4.0 分 (82.0 对 78.0)。在 Qwen3.6-Flash 上，它在 xBench-DS 上提升 7.0 分 (70.0 对 63.0)，在 AgentIF 上提升 2.9 分 (58.3 对 55.4)。两个例外体现出有界的质量–效率权衡：在 DeepSeek-V4-Flash AgentIF 上，JIT-Agent 比 Claude Code 低 3.1 分；在 Qwen3.6-Flash DeepSearchQA 上，它比 NanoBot 低 3.9 分，但两种情况下使用的词元都显著更少。没有任何固定支架能跨任务占据统治地位：例如，

表 4: 与先进智能体支架的受控比较。对于每个固定骨干模型，我们报告任务性能（性能）、每个样例的平均词元消耗（千词元，#Tokens (K)）及每个样例的 API 成本（美元）。性能越高越好；词元和成本越低越好。每个骨干模型组中的最佳值以粗体表示。

骨干	支架	DeepSearchQA			xBench-DS			AgentIF		
		性能 ↑	#Tokens (K)	成本 ↓	性能 ↑	#Tokens (K)	成本 ↓	性能 ↑	#Tokens (K)	成本 ↓
DeepSeek-V4-Flash	CLAUDE CODE	79.6	625	\$0.088	75.0	559	\$0.079	66.9	808	\$0.114
	CODEX	77.8	760	\$0.107	70.0	680	\$0.096	58.5	870	\$0.123
	OPENCODE	75.9	1,832	\$0.258	65.0	1,157	\$0.159	48.1	950	\$0.135
	HERMES	69.9	1,157	\$0.163	72.0	1,254	\$0.177	60.3	1,000	\$0.142
	NANOBOT	80.4	924	\$0.131	78.0	527	\$0.075	53.1	1,034	\$0.147
	JIT-Agent	85.1	400	\$0.066	82.0	212	\$0.039	63.8	476	\$0.097
Qwen3.6-Flash	CLAUDE CODE	72.8	710	\$0.140	58.0	650	\$0.128	55.4	900	\$0.177
	CODEX	68.5	980	\$0.193	52.0	874	\$0.172	34.2	839	\$0.170
	OPENCODE	64.1	1,968	\$0.384	44.0	1,300	\$0.256	38.7	961	\$0.217
	HERMES	60.8	1,319	\$0.256	36.0	1,155	\$0.227	49.7	1,184	\$0.223
	NANOBOT	74.2	892	\$0.197	63.0	597	\$0.119	43.5	950	\$0.187
	JIT-Agent	70.3	464	\$0.095	70.0	300	\$0.069	58.3	394	\$0.078

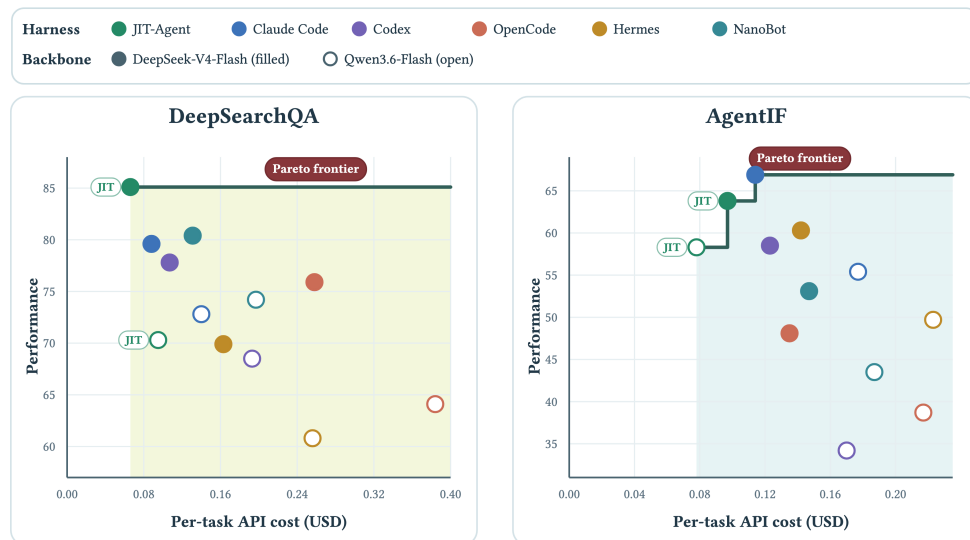


图 4: DeepSearchQA 与 AgentIF 上的成本-性能权衡。标记颜色表示支架，实心与空心圆分别表示 DeepSeek-V4-Flash 和 Qwen3.6-Flash。横轴为每个样例的 API 成本（美元），纵轴为任务性能。深绿色阶梯线勾勒全局 Pareto 前沿；浅黄绿色和蓝绿色区域中的组合至少被一个 Pareto 最优点支配。

NanoBot 在 Qwen3.6-Flash DeepSearchQA 上最强，但在 AgentIF 上落后 JIT-Agent 14.8 分。这种差异说明，应生成任务条件化支架，而不是在全局选择一个耐久脚手架。

词元与成本效率。 JIT-Agent 在全部六个受控设置中都具有最低的词元消耗与 API 成本。相对于每种设置中最便宜的固定支架，它将每个样例的成本降低 14.9–54.1%，平均降幅为 36.0%。在 DeepSeek-V4-Flash xBench-DS 上，词元消耗从 527K 降至 212K，成本从 \$0.075 降至 \$0.039，同时性能从 78.0 升至 82.0。最大成本降幅出现在 Qwen3.6-Flash AgentIF 上：JIT-Agent 使用 394K 词元、成本为 \$0.078；相比之下，固定替代方案至少使用 839K 词元、成本为 \$0.170，而 JIT-Agent 仍把最佳固定支架分数从 55.4 提升至 58.3。因此，这些收益无法用更长轨迹解释；生成的支架通常通过更短、更有选择性的执行获得更强结果。

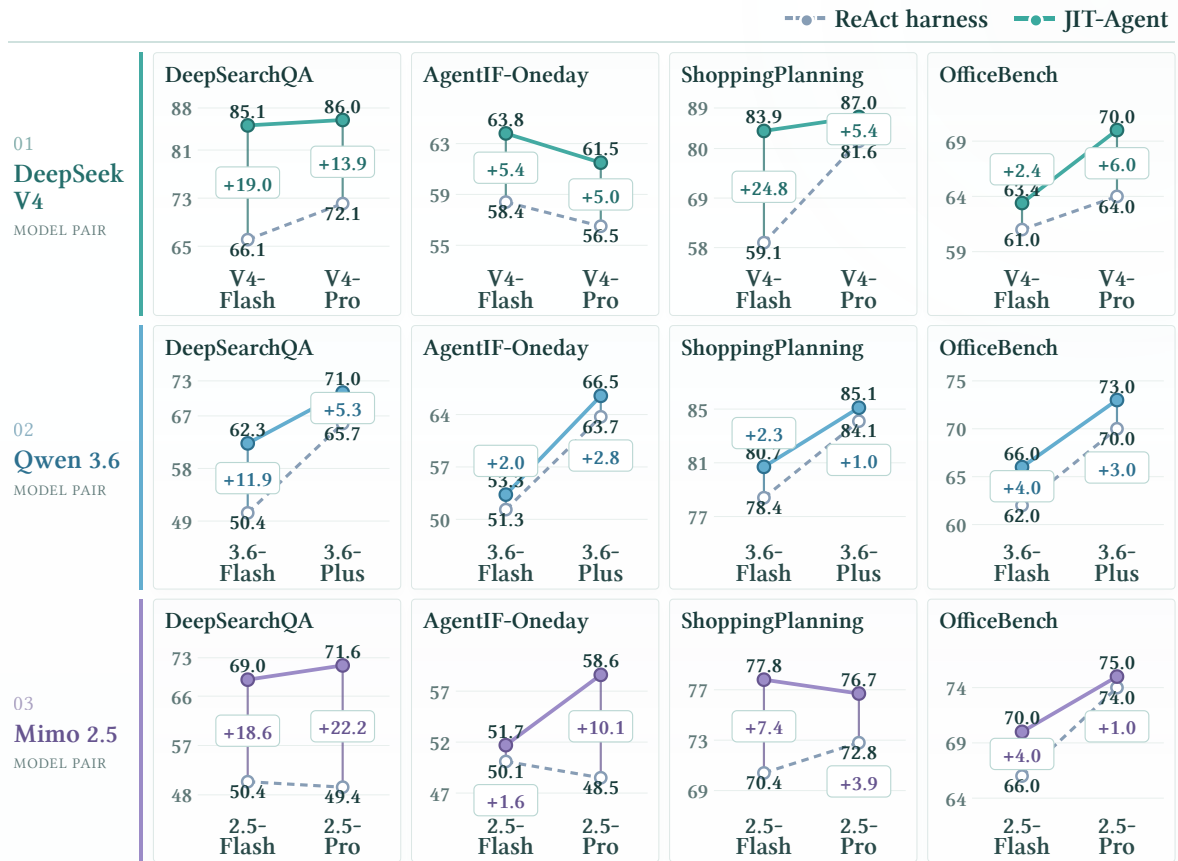


图 5: JIT 生成的支架相较 ReAct 持续改善成对骨干模型。各行对应三个模型家族及每个家族中的两个变体；各列分别对应 DeepSearchQA、AgentIF-Oneday、DeepPlanning-Shopping 和 OfficeBench。每个面板都比较同一骨干模型使用固定 ReAct 支架（虚线）和 JIT 生成支架（实线）时的结果，标注给出绝对分数增益。DeepSearchQA 使用含 100 个样例的子集；其余三个基准使用含 50 个样例的子集。

6.4 成本-性能 Pareto 前沿

图 4 可视化了 DeepSearchQA 和 AgentIF 受控比较中的成本-性能几何。每个点都是表 4 中的一个骨干-支架组合；越接近左上角的点越优。在 DeepSearchQA 上，DeepSeek-V4-Flash + JIT-Agent 是所评测组合中一个强 Pareto 最优运行点：每个样例成本为 \$0.066 时得分 85.1。与最强固定支架 NanoBot 相比，它提高 4.7 分，同时将成本降低 49.6% (\$0.131 → \$0.066)。对于 Qwen3.6-Flash，生成支架提供了计算量更低的运行点：它以 \$0.095 的成本得到 70.3 分；NanoBot 则以 \$0.197 的成本得到 74.2 分，即以 3.9 分为代价将成本降低 51.8%。AgentIF 呈现出不同但互补的前沿。在 DeepSeek-V4-Flash 上，JIT-Agent 相较 Claude Code 将成本降低 14.9% (\$0.097 对 \$0.114)，同时性能仅低 3.1 分 (63.8 对 66.9)；因此二者均位于前沿。在 Qwen3.6-Flash 上，JIT-Agent 严格支配所有固定支架：它将最佳分数从 55.4 提升至 58.3，同时把观测到的最低成本从 \$0.170 降至 \$0.078。因此，该前沿依赖任务；但 JIT 点一致向左移动表明，任务适应型支架提高了推理效率，而非通过更长轨迹购买准确率。

6.5 跨模型对的泛化

为检验 JIT 支架生成能否跨模型家族与变体迁移，我们评测了覆盖三组模型对的六个骨干模型：DeepSeek-V4-Flash/Pro、Qwen3.6-Flash/Plus 和 MIMO-V2.5-Flash/Pro。对于每个骨干模型，我们固定模型，并在 DeepSearchQA、AgentIF-Oneday、DeepPlanning-Shopping 和 OfficeBench 上比较其标准 ReAct 支架与 JIT 生成的支架。

在全部 24 个直接匹配的骨干-基准比较中，JIT 生成的支架均优于 ReAct，平均提高 7.6 分。该效



图 6: 跨任务流的流式测试时支架演化。图中给出 DeepPlanning-Shopping、DeepPlanning-Travel 和 OfficeBench 上的累积准确率（上）、每任务 API 成本（中）和每任务工具调用次数（下）。虚线表示静态 JIT，其中任务专用支架的生成彼此独立；实线表示流式 JIT，它会随着新任务到来持续吸收执行反馈。阴影区域表示流式轨迹周围的局部变化。流式 JIT 在三个基准结束时都获得更高的累积准确率，而 API 成本与工具使用轨迹仍依赖具体任务，且总体量级相近。

果对每个家族及每组模型对中的两个变体都成立：DeepSeek V4、Qwen 3.6 和 Mimo 2.5 的平均增益分别为 10.2、4.0 和 8.6 分。DeepSearchQA 的平均增益最大，达到 15.2 分，其中 Mimo-V2.5-Pro 和 DeepSeek-V4-Flash 分别提高 22.2 和 19.0 分；DeepPlanning-Shopping 平均提高 7.5 分，其中 DeepSeek-V4-Flash 提高 24.8 分。这些在同一骨干模型内的一致增益表明，支架智能能够跨模型家族和变体迁移，而非仅仅弥补某一特定骨干模型的不足。

6.6 测试时支架演化

JIT-Agent 不仅用于一次性生成支架，还会根据执行反馈改进其档案。我们比较了支架生成彼此独立的静态 JIT，以及在整个评测流中持续检索和更新支架的流式 JIT。

流式 JIT 在 DeepPlanning-Shopping、DeepPlanning-Travel 和 OfficeBench 上的最终结果均高于静态变体。在三个任务流中，累积准确率优势都随着执行反馈的积累而出现，并一直保持到评测结束。相应的成本和工具调用轨迹表明，最终收益并非都与更大的交互预算绑定。这一模式符合 Evo-GDPO 的目标：只有推进档案前沿的支架才会被保留。

6.7 生成支架的可视化

定量结果已经表明，生成的支架能够改善执行；接下来我们考察 JIT-Agent 实际生成的内容。图 7 和图 8 可视化了在同一四模块协议下，为计算结构截然不同的任务生成的两个支架。二者的差异不止在于提示或模块名称：生成的规划表示、动作拓扑、持久状态和暴露能力都围绕每项任务的需求重新组织。

把工件生产表示为依赖图。 PALIMPSEST 面向一项跨应用任务生成；该任务必须发现联系人卡片、排除一个人、对其余记录进行规范化和排序、创建工作簿，并通过电子邮件交付。支架把这些要求编

Palimpsest — graph-planned artifact execution

A task becomes a dependency graph, executes with bounded parallelism, and leaves reusable state.

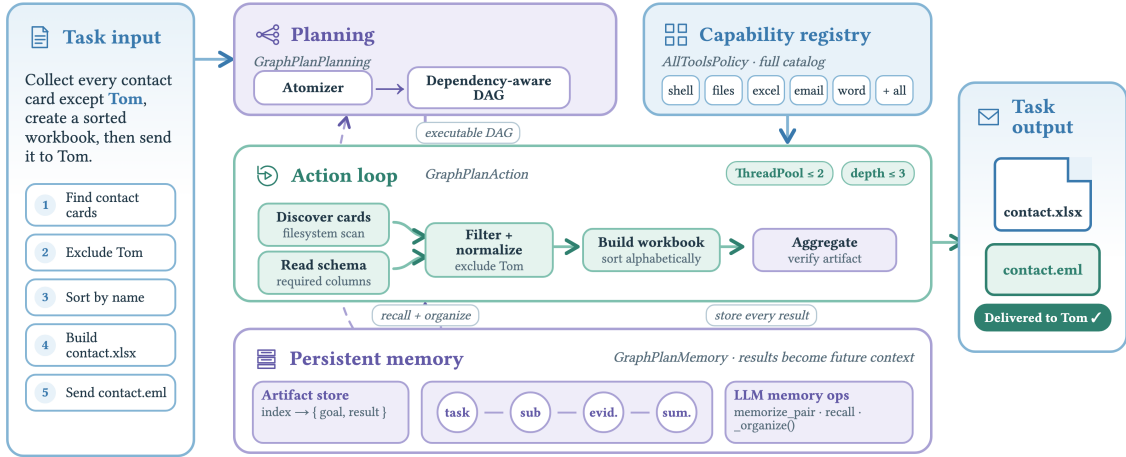


图 7: Palimpsest: 由图规划的工件执行。GraphPlanPlanning 将联系人处理请求转化为 DAG; GraphPlanAction 以有界宽度和深度执行该图, 而 GraphPlanMemory 存储可复用工件和推理状态。

Trapdoor — delegation as a runtime primitive

A synthetic tool call opens a bounded sub-agent; its answer returns through the ordinary observation loop.

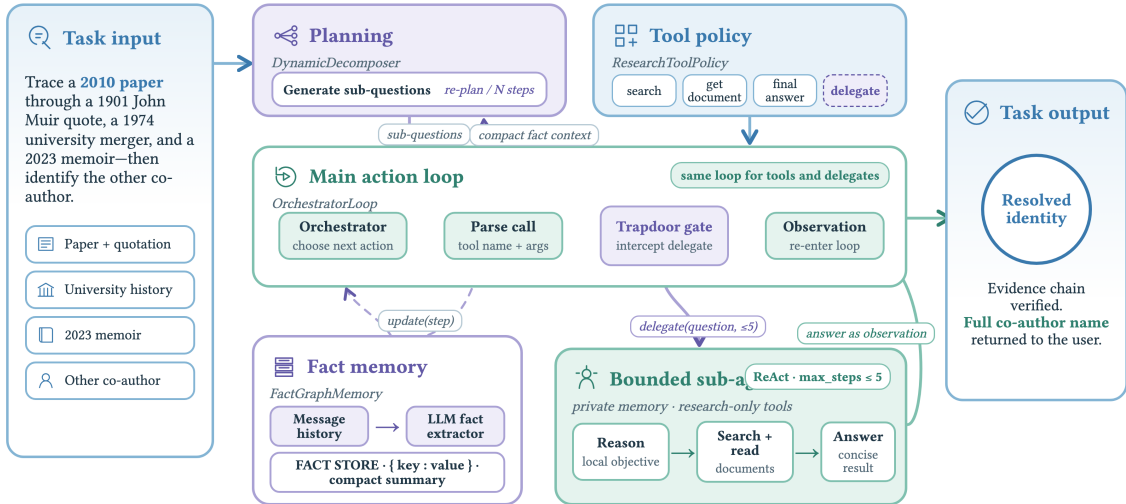


图 8: Trapdoor: 委派工具调用背后的有界研究。合成的 delegate 能力由 OrchestratorLoop 截获; 后者运行有界子智能体, 并把提取的事实写入 FactGraphMemory。

译为显式 DAG: 发现与模式检查为筛选提供前置条件, 工作簿构建等待规范化记录, 而交付则等待工件验证。随后, GraphPlanAction 以有界宽度和深度执行就绪节点; GraphPlanMemory 则为中间目标、证据和工件建立索引, 使下游节点能够直接使用已完成结果, 而无需从交互记录中重建它们。所得支架匹配任务的工件依赖与提交顺序, 把冗长的自然语言请求转化为可验证的生产流水线。

把深度研究表示为有界递归委派。 TRAPDOOR 处理一个多跳身份问题, 其线索横跨一段历史引文、一次大学合并、一本回忆录和一篇研究论文。固定 DAG 会过早承诺一条尚不确定的证据路径, 因此生成的支架使用 DynamicDecomposer 形成并修订子问题, 并在研究工具策略中增添合成的 delegate 能力。一次 delegate 调用会打开私有研究子智能体; 该子智能体拥有自己的记忆、仅限研究的工具及五步预算, 返回的答案再作为普通观察进入父循环。与此同时, FactGraphMemory 从不断增长的

历史中提取紧凑的键-值事实，使父智能体无需继承每条子智能体轨迹，就能整合各分支证据。因此，对于不确定性需要通过分支式证据收集来解决的任务，该支架专门把递归研究变成一种运行时原语。

这种对比构成了核心定性结果：同一个生成器把一项任务映射为带工件存储的图执行，把另一项任务映射为带事实存储的递归编排。共享协议约束的是接口，而非行为。附录 A 中的其他案例把这一模式扩展到层次化上下文折叠、完成度门控的工具访问、阶段条件化执行、证据矩阵搜索、选择性上下文渲染、有类型计算状态、逐文件确定性验证及局部动作修复；其流程见图 A.1–A.8。

7 结论与未来工作

结论。我们提出了 JIT-Agent，一种专门在推理时合成、修复和演化任务条件化智能体支架的模型。通过用可组合四模块协议表示支架，并借助定制学习、修复监督和 Evo-GDPO 训练生成器，JIT-Agent 把支架构建从人工工程化的预先工件转化为学习所得能力。在深度研究、日常工作、规划和工作区任务上，JIT 生成的支架持续增强其底层骨干模型，与前沿模型和先进固定支架竞争，并改善成本-性能权衡。这些结果确立了支架智能，也就是适配模型赖以行动的运行脚手架的能力；它是超越模型权重本身、可训练且可迁移的智能体能力来源。

未来工作。我们把支架智能视为与模型容量和推理算力并列的另一个基础扩展维度，并将 JIT-Agent 视为实现这一愿景的一步。长期机遇在于模型-支架协同设计范式：基础模型与塑造其记忆、规划、动作和工具使用方式的运行结构共同训练。尽管如此，我们认为未来系统不必遵循本文刻意探索的激进形式，即整个脚手架均可即时重新设计；生产运行时可以保留稳定核心，同时允许模型在任务需要时构建、修订或替换选定的支架组件。随着这一范式成熟，本文为支架合成、修复与演化开发的能力将越来越多地被基础模型自身内化。届时，模型不仅会学习如何在给定支架内行动，还会学习如何改进其赖以行动的支架，从而围绕适应性接口、可验证运行时修改及模型与执行系统的协同扩展，开启广阔的研究议程。

贡献者

项目负责人 Guibin Zhang

核心贡献者 Guibin Zhang, Leo Lu, Fangzhou Xie

贡献者 Kang Zhu, Junhao Wang, Zhifei Xie, Zhaochen Yu, Zihang Liu, Zhongxiang Sun, Qiankun Li, Yue Liao, Heng Chang, Xiaobin Hu, Qibing Ren

通讯作者 Wangchunshu Zhou, Shuicheng Yan

参考文献

Salaheddin Alzu'bi, Baran Nama, Arda Kaz, Anushri Eswaran, Weiyuan Chen, Sarvesh Khetan, Rishab Bala, Tu Vu, and Sewoong Oh. Roma: Recursive open meta-agent framework for long-horizon multi-agent systems, 2026. URL <https://arxiv.org/abs/2602.01848>.

Anthropics. GitHub - anthropics/claude-code: Claude Code is an agentic coding tool that lives in your terminal, understands your codebase, and helps you code faster by executing routine tasks, explaining complex code, and handling git workflows - all through natural language commands. — github.com. <https://github.com/anthropics/claude-code>, 2025. [Accessed 04-01-2026].

Apodex Team. Apodex-1.0: Self-evolving heavy-duty solver. <https://www.apodex.com/blog/apodex-1.0>, 2026.

Fei Bai, Huatong Song, Shuang Sun, Daixuan Cheng, Yike Yang, Chuan Hao, Renyuan Li, Feng Chang, Yuan Wei, Ran Tao, Bryan Dai, Jian Yang, Wayne Xin Zhao, and Ji-Rong Wen. Clawgym: A scalable framework for building effective claw agents, 2026. URL <https://arxiv.org/abs/2604.26904>.

- James Burgess, Jan N. Hansen, Duo Peng, Yuhui Zhang, Alejandro Lozano, Min Woo Sun, Emma Lundberg, and Serena Yeung-Levy. Papersearchqa: Learning to search and reason over scientific papers with rlvr, 2026. URL <https://arxiv.org/abs/2601.18207>.
- Pengfei Cao, Tianyi Men, Wencan Liu, Jingwen Zhang, Xuzhao Li, Xixun Lin, Dianbo Sui, Yanan Cao, Kang Liu, and Jun Zhao. Large language models for planning: A comprehensive and systematic survey, 2025. URL <https://arxiv.org/abs/2505.19683>.
- Angelica Chen, David Dohan, and David So. Evoprompting: Language models for code-level neural architecture search. *Advances in neural information processing systems*, 36:7787–7817, 2023.
- Kaiyuan Chen, Yixin Ren, Yang Liu, Xiaobo Hu, Haotong Tian, Tianbao Xie, Fangfu Liu, Haoye Zhang, Hongzhang Liu, Yuan Gong, Chen Sun, Han Hou, Hui Yang, James Pan, Jianan Lou, Jiayi Mao, Jizheng Liu, Jinpeng Li, Kangyi Liu, Kenkun Liu, Rui Wang, Run Li, Tong Niu, Wenlong Zhang, Wenqi Yan, Xuanzheng Wang, Yuchen Zhang, Yi-Hsin Hung, Yuan Jiang, Zexuan Liu, Zihan Yin, Zijian Ma, and Zhiwen Mo. xbench: Tracking agents productivity scaling with profession-aligned real-world evaluations, 2025a. URL <https://arxiv.org/abs/2506.13651>.
- Kaiyuan Chen, Qimin Wu, Taiyu Hou, Tianhao Tang, Xueyu Hu, Yuchen Hou, Bikun Li, Chengming Qian, Guoyin Wang, Haolin Chen, Haotong Tian, Haoye Zhang, Haoyu Bian, Hongbing Pan, Hongkang Zhang, Hongyi Zhou, Jiaqi Cai, Jiewu Rao, Jiyan Ren, Keduan Huang, Lucia Zhu Huang, Mingyu Yuan, Naixu Guo, Qicheng Tang, Qinyan Zhang, Shuai Chen, Siheng Chen, Ting Ting Li, Xiaoxing Guo, Yaocheng Zuo, Yaoqi Guo, Yinan Wang, Yinzhou Yu, Yize Wang, Yuan Jiang, Yuan Tian, Yuanshuo Zhang, Yuxuan Liu, Yvette Yan Zeng, Zenyu Shan, Zihan Yin, Xiaobo Hu, Yang Liu, Yixin Ren, and Yuan Gong. Agentif-oneday: A task-level instruction-following benchmark for general ai agents in daily scenarios, 2026a. URL <https://arxiv.org/abs/2601.20613>.
- Tingyang Chen, Shuo Lu, Kang Zhao, Weicheng Meng, Hanlin Teng, Tianhao Li, Chao Li, Xule Liu, Jian Liang, Zhizhong Zhang, Yuan Xie, Heng Qu, Kun Shao, and Jian Luan. Harnessx: A composable, adaptive, and evolvable agent harness foundry, 2026b. URL <https://arxiv.org/abs/2606.14249>.
- Zijian Chen, Xueguang Ma, Shengyao Zhuang, Ping Nie, Kai Zou, Andrew Liu, Joshua Green, Kshama Patel, Ruoxi Meng, Mingyi Su, Sahel Sharifymoghaddam, Yanxi Li, Haoran Hong, Xinyu Shi, Xuye Liu, Nandan Thakur, Crystina Zhang, Luyu Gao, Wenhui Chen, and Jimmy Lin. Browsecomp-plus: A more fair and transparent evaluation benchmark of deep-research agent, 2025b. URL <https://arxiv.org/abs/2508.06600>.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory, 2025. URL <https://arxiv.org/abs/2504.19413>.
- ClaudeCode. Agent Skills - Claude Code Docs — code.claude.com. <https://code.claude.com/docs/en/skills>, 2025. [Accessed 03-01-2026].
- DeepSeek-AI. Deepseek-v4 preview release. <https://api-docs.deepseek.com/news/news260424/>, 2026.
- Lutfi Eren Erdogan, Nicholas Lee, Sehoon Kim, Suhong Moon, Hiroki Furuta, Gopala Anumanchipalli, Kurt Keutzer, and Amir Gholami. Plan-and-act: Improving planning of agents for long-horizon tasks. *arXiv preprint arXiv:2503.09572*, 2025a.
- Lutfi Eren Erdogan, Nicholas Lee, Sehoon Kim, Suhong Moon, Hiroki Furuta, Gopala Anumanchipalli, Kurt Keutzer, and Amir Gholami. Plan-and-act: Improving planning of agents for long-horizon tasks, 2025b. URL <https://arxiv.org/abs/2503.09572>.

- Google DeepMind. Gemini 3.1 pro model card. <https://deepmind.google/models/model-cards/gemini-3-1-pro>, 2026a.
- Google DeepMind. Gemini 3.5 flash model card. <https://deepmind.google/models/model-cards/gemini-3-5-flash/>, 2026b.
- Nikita Gupta, Riju Chatterjee, Lukas Haas, Connie Tao, Andrew Wang, Chang Liu, Hidekazu Oiwa, Elena Gribovskaya, Jan Ackermann, John Blitzer, Sasha Goldshtein, and Dipanjan Das. Deepsearchqa: Bridging the comprehensiveness gap for deep research agents, 2026. URL <https://arxiv.org/abs/2601.20975>.
- HKUDS. Nanobot: Lightweight open-source ai agent. <https://github.com/HKUDS/nanobot>, 2026.
- Chuanrui Hu, Xingze Gao, Zuyi Zhou, Dannong Xu, Yi Bai, Xintong Li, Hui Zhang, Tong Li, Chong Zhang, Lidong Bing, and Yafeng Deng. Evermemos: A self-organizing memory operating system for structured long-horizon reasoning, 2026a. URL <https://arxiv.org/abs/2601.02163>.
- Mengkang Hu, Tianxing Chen, Qiguang Chen, Yao Mu, Wenqi Shao, and Ping Luo. Hiagent: Hierarchical working memory management for solving long-horizon agent tasks with large language model, 2024a. URL <https://arxiv.org/abs/2408.09559>.
- Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*, 2024b.
- Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, Jiahang Lin, Honglin Guo, Shihan Dou, Zhiheng Xi, Senjie Jin, Jiejun Tan, Yanbin Yin, Jiongnan Liu, Zeyu Zhang, Zhongxiang Sun, Yutao Zhu, Hao Sun, Boci Peng, Zhenrong Cheng, Xuanbo Fan, Jiaxin Guo, Xinlei Yu, Zhenhong Zhou, Zewen Hu, Jiahao Huo, Junhao Wang, Yuwei Niu, Yu Wang, Zhenfei Yin, Xiaobin Hu, Yue Liao, Qiankun Li, Kun Wang, Wangchunshu Zhou, Yixin Liu, Dawei Cheng, Qi Zhang, Tao Gui, Shirui Pan, Yan Zhang, Philip Torr, Zhicheng Dou, Ji-Rong Wen, Xu-anjing Huang, Yu-Gang Jiang, and Shuicheng Yan. Memory in the age of ai agents, 2025. URL <https://arxiv.org/abs/2512.13564>.
- Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, Jiahang Lin, Honglin Guo, Shihan Dou, Zhiheng Xi, Senjie Jin, Jiejun Tan, Yanbin Yin, Jiongnan Liu, Zeyu Zhang, Zhongxiang Sun, Yutao Zhu, Hao Sun, Boci Peng, Zhenrong Cheng, Xuanbo Fan, Jiaxin Guo, Xinlei Yu, Zhenhong Zhou, Zewen Hu, Jiahao Huo, Junhao Wang, Yuwei Niu, Yu Wang, Zhenfei Yin, Xiaobin Hu, Yue Liao, Qiankun Li, Kun Wang, Wangchunshu Zhou, Yixin Liu, Dawei Cheng, Qi Zhang, Tao Gui, Shirui Pan, Yan Zhang, Philip Torr, Zhicheng Dou, Ji-Rong Wen, Xu-anjing Huang, Yu-Gang Jiang, and Shuicheng Yan. Memory in the age of ai agents, 2026b. URL <https://arxiv.org/abs/2512.13564>.
- Jiajie Jin, Xiaoxi Li, Guanting Dong, Yuyao Zhang, Yutao Zhu, Yang Zhao, Hongjin Qian, and Zhicheng Dou. Hira: A hierarchical reasoning framework for decoupled planning and execution in deep search, 2025. URL <https://arxiv.org/abs/2507.02652>.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Moazam, et al. Dspy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023.
- LangGraph. GitHub - langchain-ai/langgraph: Build resilient language agents as graphs. — github.com. <https://github.com/langchain-ai/langgraph>, 2025. [Accessed 03-01-2026].

- Hyunin Lee, Jinglue Xu, Jeffrey Seely, Donghyun Lee, Matei Zaharia, and Yujin Tang. Recursive harness self-improvement, 2026a. URL <https://arxiv.org/abs/2607.15524>.
- Sanghoon Lee, Jiyeong Chae, and Kyung-Joon Park. Harness engineering for physical ai: Robot middleware is the harness layer, 2026b. URL <https://arxiv.org/abs/2606.09416>.
- Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses, 2026c. URL <https://arxiv.org/abs/2603.28052>.
- Yoonsang Lee, Howard Yen, Xi Ye, and Danqi Chen. Agentic aggregation for parallel scaling of long-horizon agentic tasks, 2026d. URL <https://arxiv.org/abs/2604.11753>.
- Hao Li, Chunjiang Mu, Jianhao Chen, Siyue Ren, Zhiyao Cui, Yiqun Zhang, Lei Bai, and Shuyue Hu. Organizing, orchestrating, and benchmarking agent skills at ecosystem scale, 2026a. URL <https://arxiv.org/abs/2603.02176>.
- Xiaoxi Li, Wenxiang Jiao, Jiarui Jin, Guanting Dong, Jiajie Jin, Yinuo Wang, Hao Wang, Yutao Zhu, Ji-Rong Wen, Yuan Lu, and Zhicheng Dou. Deepagent: A general reasoning agent with scalable toolsets, 2026b. URL <https://arxiv.org/abs/2510.21618>.
- Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, and Yu-Gang Jiang. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses, 2026. URL <https://arxiv.org/abs/2604.25850>.
- Jiaxi Liu, Yanzuo Jiang, Guibin Zhang, Zihan Zhang, Heng Chang, Zhenfei Yin, Qibing Ren, and Junchi Yan. Todoevolve: Learning to architect agent planning systems, 2026a. URL <https://arxiv.org/abs/2602.07839>.
- Shih-Yang Liu, Xin Dong, Ximing Lu, Shizhe Diao, Peter Belcak, Mingjie Liu, Min-Hung Chen, Hongxu Yin, Yu-Chiang Frank Wang, Kwang-Ting Cheng, Yejin Choi, Jan Kautz, and Pavlo Molchanov. Gdpo: Group reward-decoupled normalization policy optimization for multi-reward rl optimization, 2026b. URL <https://arxiv.org/abs/2601.05242>.
- Zewen Liu, Zhan Shi, Yisi Sang, Bing He, Minhua Lin, Tianxin Wei, Dakuo Wang, Benoit Dumoulin, Wei Jin, and Hanqing Lu. Adaptive auto-harness: Sustained self-improvement for agentic system deployment on open-ended task streams, 2026c. URL <https://arxiv.org/abs/2606.01770>.
- Xinghua Lou, Miguel Lázaro-Gredilla, Antoine Dedieu, Carter Wendelken, Wolfgang Lehrach, and Kevin P. Murphy. AutoHarness: Improving llm agents by automatically synthesizing a code harness, 2026. URL <https://arxiv.org/abs/2603.03329>.
- Junyu Luo, Weizhi Zhang, Ye Yuan, Yusheng Zhao, Junwei Yang, Yiyang Gu, Bohan Wu, Binqi Chen, Ziyue Qiao, Qingqing Long, Rongcheng Tu, Xiao Luo, Wei Ju, Zhiping Xiao, Yifan Wang, Meng Xiao, Chenwu Liu, Jingyang Yuan, Shichang Zhang, Yiqiao Jin, Fan Zhang, Xian Wu, Hanqing Zhao, Dacheng Tao, Philip S. Yu, and Ming Zhang. Large language model agent: A survey on methodology, applications and challenges, 2025. URL <https://arxiv.org/abs/2503.21460>.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Sean Welleck, Bodhisattwa Prasad Majumder, Shashank Gupta, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback, 2023.

- Qianyu Meng, Yanan Wang, Liyi Chen, Qimeng Wang, Chengqiang Lu, Wei Wu, Yan Gao, Yi Wu, and Yao Hu. Agent harness for large language model agents: A survey. *Preprints*, April 2026a. doi: 10.20944/preprints202604.0428.v2. URL <https://doi.org/10.20944/preprints202604.0428.v2>.
- Qianyu Meng, Yanan Wang, Liyi Chen, Qimeng Wang, Chengqiang Lu, Wei Wu, Yan Gao, Yi Wu, and Yao Hu. Agent harness for large language model agents: A survey. 2026b.
- Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Jesse Hu, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026. URL <https://arxiv.org/abs/2601.11868>.
- Microsoft. GitHub - ag2ai/ag2: AG2 (formerly AutoGen): The Open-Source AgentOS. <https://github.com/ag2ai/ag2>, 2024. [Accessed 31-08-2025].
- Moonshot AI. Kimi k2.7 code: Open-source agentic coding model. <https://www.kimi.com/resources/kimi-k2-7-code>, 2026.
- Jun Nie, Yonggang Zhang, Qianshu Cai, Yiu-ming Cheung, Xinmei Tian, and Bo Han. EvolveNet: Collaborative harness evolution for agent self-improvement, 2026a. URL <https://arxiv.org/abs/2608.04968>.
- Jun Nie, Yonggang Zhang, Jun Song, Qianshu Cai, Dahai Yu, Yike Guo, Xinmei Tian, and Bo Han. TTHE: Test-time harness evolution, 2026b. URL <https://arxiv.org/abs/2607.08124>.
- Xuying Ning, Katherine Tieu, Dongqi Fu, Tianxin Wei, Zihao Li, Yuanchen Bei, Jiaru Zou, Mengting Ai, Zhining Liu, Ting-Wei Li, et al. Code as agent harness. *arXiv preprint arXiv:2605.18747*, 2026.
- Nous Research. Hermes agent. <https://github.com/NousResearch/hermes-agent>, 2026.
- OpenAI. Codex | AI Coding Partner from OpenAI — openai.com. <https://openai.com/codex/>, 2025.
- OpenAI. Gpt-5.6: Frontier intelligence that scales with your ambition. <https://openai.com/index/gpt-5-6/>, 2026.
- OpenClaw Contributors. Openclaw: Your own personal ai assistant. <https://github.com/openclaw/openclaw>, 2026.
- OpenCode-AI. GitHub - opencode-ai/opencode: A powerful AI coding agent. Built for the terminal. — github.com. <https://github.com/opencode-ai/opencode>, 2025.
- Linyue Pan, Lexiao Zou, Shuo Guo, Jingchen Ni, and Hai-Tao Zheng. Natural-language agent harnesses. *arXiv preprint arXiv:2603.25723*, 2026a.

- Wenbo Pan, Shujie Liu, Chin-Yew Lin, Jingying Zeng, Xianfeng Tang, Xiangyang Zhou, Yan Lu, and Xiaohua Jia. Evolving agents in the dark: Retrospective harness optimization via self-preference, 2026b. URL <https://arxiv.org/abs/2606.05922>.
- PinchBench. Best Models by Success Rate | PinchBench — pinchbench.com. <https://pinchbench.com/>, 2026. [Accessed 11-04-2026].
- Cheng Qian, Emre Can Acikgoz, Qi He, Hongru Wang, Xiusi Chen, Dilek Hakkani-Tür, Gokhan Tur, and Heng Ji. Toolrl: Reward is all tool learning needs. *arXiv preprint arXiv:2504.13958*, 2025.
- Hongjin Qian, Zhao Cao, and Zheng Liu. Memobrain: Executive memory as an agentic brain for reasoning, 2026. URL <https://arxiv.org/abs/2601.08079>.
- Tianrui Qin, Qianben Chen, Sinuo Wang, He Xing, King Zhu, He Zhu, Dingfeng Shi, Xinxin Liu, Ge Zhang, Jiaheng Liu, Yuchen Eleanor Jiang, Xitong Gao, and Wangchunshu Zhou. Flash-searcher: Fast and effective web agents via dag-based parallel execution, 2025. URL <https://arxiv.org/abs/2509.25301>.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. Toolllm: Facilitating large language models to master 16000+ real-world apis, 2023. URL <https://arxiv.org/abs/2307.16789>.
- Qwen Team. Qwen3.7-plus: Multimodal agent intelligence. <https://qwen.ai/home>, 2026.
- Jianhao Ruan, Zhihao Xu, Yiran Peng, Fashen Ren, Zhaoyang Yu, Xinbing Liang, Jinyu Xiang, Yongru Chen, Bang Liu, Chenglin Wu, Yuyu Luo, and Jiayi Zhang. Aorchestra: Automating sub-agent creation for agentic orchestration, 2026. URL <https://arxiv.org/abs/2602.03786>.
- Anjana Sarkar and Soumyendu Sarkar. Survey of llm agent communication with mcp: A software design pattern centric review, 2025. URL <https://arxiv.org/abs/2506.05364>.
- Yu Shang, Yu Li, Keyu Zhao, Likai Ma, Jiahe Liu, Fengli Xu, and Yong Li. Agentsquare: Automatic llm agent search in modular design space. *arXiv preprint arXiv:2410.06153*, 2024.
- Shuai Shao, Kangning Zhang, Qingyao Li, Shijian Wang, Hao Wang, Wenxiang Jiao, Yuan Lu, Yi Guo, Weiwen Liu, and Weinan Zhang. Harness-R1: Learning to edit executable runtime harnesses from agent failure trajectories, 2026. URL <https://arxiv.org/abs/2608.02276>.
- Dingfeng Shi, Jingyi Cao, Qianben Chen, Weichen Sun, Weizhen Li, Hongxuan Lu, Fangchen Dong, Tianrui Qin, King Zhu, Minghao Liu, Jian Yang, Ge Zhang, Jiaheng Liu, Changwang Zhang, Jun Wang, Yuchen Eleanor Jiang, and Wangchunshu Zhou. Taskcraft: Automated generation of agentic tasks, 2025. URL <https://arxiv.org/abs/2506.10055>.
- Yifan Shi, Wei Zhang, and Tianyi Cui. A programming paradigm for spatiotemporal composability, 2026. URL <https://github.com/cordiverse/paper>. Preprint, draft of August 13, 2026.
- Noah Shinn, Beck Labash, and Ashwin Gopinath. Reflexion: an autonomous agent with dynamic memory and self-reflection. *arXiv preprint*, abs/2303.11366, 2023. doi: 10.48550/arXiv.2303.11366. URL <https://doi.org/10.48550/arXiv.2303.11366>.
- Xiaoshuai Song, Haofei Chang, Guanting Dong, Yutao Zhu, Ji-Rong Wen, and Zhicheng Dou. Envs-cal: Scaling tool-interactive environments for llm agent via programmatic synthesis, 2026. URL <https://arxiv.org/abs/2601.05808>.

- Haotian Sun, Yuchen Zhuang, Ling kai Kong, Bo Dai, and Chao Zhang. Adaplaner: Adaptive planning from feedback with language models. *Advances in neural information processing systems*, 36: 58202–58245, 2023.
- Hongru Wang, Cheng Qian, Wan jun Zhong, Xiusi Chen, Jiahao Qiu, Shijue Huang, Bowen Jin, Mengdi Wang, Kam-Fai Wong, and Heng Ji. Otc: Optimal tool calls via reinforcement learning. *arXiv preprint arXiv:2504.14870*, 2025a.
- Weixuan Wang, Dongge Han, Daniel Madrigal Diaz, Jin Xu, Victor Rühle, and Saravan Rajmohan. Odysseybench: Evaluating llm agents on long-horizon complex office application workflows, 2025b. URL <https://arxiv.org/abs/2508.09124>.
- Zilong Wang, Yuedong Cui, Li Zhong, Zimin Zhang, Da Yin, Bill Yuchen Lin, and Jingbo Shang. Officebench: Benchmarking language agents across multiple applications for office automation, 2024. URL <https://arxiv.org/abs/2407.19056>.
- Jialong Wu, Wenbiao Yin, Yong Jiang, Zhenglin Wang, Zekun Xi, Runnan Fang, Linhai Zhang, Yulan He, Deyu Zhou, Pengjun Xie, and Fei Huang. Webwalker: Benchmarking llms in web traversal, 2025. URL <https://arxiv.org/abs/2501.07572>.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation framework, August 01, 2023 2023.
- Xixi Wu, Kuan Li, Yida Zhao, Liwen Zhang, Litu Ou, Huifeng Yin, Zhongwang Zhang, Xinmiao Yu, Dingchu Zhang, Yong Jiang, Pengjun Xie, Fei Huang, Minhao Cheng, Shuai Wang, Hong Cheng, and Jingren Zhou. Resum: Unlocking long-horizon search intelligence via context summarization, 2026. URL <https://arxiv.org/abs/2509.13313>.
- Renjun Xu and Yang Yan. Agent skills for large language models: Architecture, acquisition, security, and the path forward, 2026. URL <https://arxiv.org/abs/2602.12430>.
- B. Y. Yan, Chaofan Li, Hongjin Qian, Shuqi Lu, and Zheng Liu. General agentic memory via deep research, 2025. URL <https://arxiv.org/abs/2511.18423>.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering, 2024. URL <https://arxiv.org/abs/2405.15793>.
- Xiaofang Yang, Lijun Li, Heng Zhou, Tong Zhu, Xiaoye Qu, Yuchen Fan, Qianshan Wei, Rui Ye, Li Kang, Yiran Qin, Zhiqiang Kou, Daizong Liu, Qi Li, Ning Ding, Siheng Chen, and Jing Shao. Toward efficient agents: Memory, tool learning, and planning, 2026. URL <https://arxiv.org/abs/2601.14192>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023a. URL <https://arxiv.org/abs/2210.03629>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2023b.
- Rui Ye, Zhongwang Zhang, Kuan Li, Huifeng Yin, Zhengwei Tao, Yida Zhao, Liangcai Su, Liwen Zhang, Zile Qiao, Xinyu Wang, Pengjun Xie, Fei Huang, Siheng Chen, Jingren Zhou, and Yong Jiang. Agentfold: Long-horizon web agents with proactive context management, 2025. URL <https://arxiv.org/abs/2510.24699>.

- Siyu Yuan, Kaitao Song, Jiangjie Chen, Xu Tan, Dongsheng Li, and Deqing Yang. Evoagent: Towards automatic multi-agent generation via evolutionary algorithms. *arXiv preprint arXiv:2406.14228*, 2024.
- Mert Yuksekogonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. Textgrad: Automatic ”differentiation” via text, 2024. URL <https://arxiv.org/abs/2406.07496>.
- Z.ai. Glm-5.2: Built for long-horizon tasks. <https://z.ai/blog/glm-5.2>, 2026.
- Alex L. Zhang, Tim Kraska, and Omar Khattab. Recursive language models, 2026a. URL <https://arxiv.org/abs/2512.24601>.
- Guibin Zhang, Luyang Niu, Junfeng Fang, Kun Wang, Lei Bai, and Xiang Wang. Multi-agent architecture search via agentic supernet. *arXiv preprint arXiv:2502.04180*, 2025a.
- Guibin Zhang, Haotian Ren, Chong Zhan, Zhenhong Zhou, Junhao Wang, He Zhu, Wangchunshu Zhou, and Shuicheng Yan. Memevolve: Meta-evolution of agent memory systems, 2025b. URL <https://arxiv.org/abs/2512.18746>.
- Hangfan Zhang, Shao Zhang, Kangcong Li, Chen Zhang, Yang Chen, Yiqun Zhang, Lei Bai, and Shuyue Hu. Self-harness: Harnesses that improve themselves, 2026b. URL <https://arxiv.org/abs/2606.09498>.
- Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. AFlow: Automating Agentic Workflow Generation, October 2024. URL <http://arxiv.org/abs/2410.10762>. arXiv:2410.10762.
- Luan Zhang, Ruochen Zhou, Dandan Song, Zhengyu Chen, Yuhang Tian, Jun Yang, Huipeng Ma, Chenhao Li, Guangyuan Feng, Xudong Li, Yizhou Jin, and Yan Xu. HarnessCompass: Guiding automatic harness evolution toward generalizable and effective agent harnesses, 2026c. URL <https://arxiv.org/abs/2608.01918>.
- Yinger Zhang, Shutong Jiang, Renhao Li, Jianhong Tu, Yang Su, Lianghao Deng, Xudong Guo, Chenxu Lv, and Junyang Lin. Deepplanning: Benchmarking long-horizon agentic planning with verifiable constraints, 2026d. URL <https://arxiv.org/abs/2601.18137>.
- Yixian Zhang, Huanming Zhang, Feng Gao, Xiao Li, Zhihao Liu, Chunyang Zhu, Jiaying Qiu, Yuchen Yan, Jiayuan Liu, Wenhao Tang, Zhengru Fang, Yi Nie, Changxu Wei, Yu Wang, Wenbo Ding, and Chao Yu. Harness vla: Steering frozen vlas into reliable manipulation primitives via memory-guided agents, 2026e. URL <https://arxiv.org/abs/2607.08448>.
- Yuxiang Zhang, Jiangming Shu, Ye Ma, Xueyuan Lin, Shangxi Wu, and Jitao Sang. Memory as action: Autonomous context curation for long-horizon agentic tasks, 2025c. URL <https://arxiv.org/abs/2510.12635>.
- Boyuan Zheng, Michael Y. Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, and Yu Su. Skillweaver: Web agents can self-improve by discovering and honing skills, 2025. URL <https://arxiv.org/abs/2504.07079>.
- Chenyu Zhou, Huacan Chai, Wenteng Chen, Zihan Guo, Rong Shan, Yuanyi Song, Tianyi Xu, Yingxuan Yang, Aofan Yu, Weiming Zhang, Congming Zheng, Jiachen Zhu, Zeyu Zheng, Zhuosheng Zhang, Xingyu Lou, Changwang Zhang, Zhihui Fu, Jun Wang, Weiwen Liu, Jianghao Lin, and Weinan Zhang. Externalization in llm agents: A unified review of memory, skills, protocols and harness engineering, 2026. URL <https://arxiv.org/abs/2604.08224>.

Tailin Zhou. Hierarchical self-improvement: A framework for task-specific evolvable agent harnesses, 2026. URL <https://arxiv.org/abs/2608.08466>.

He Zhu, Tianrui Qin, King Zhu, Heyuan Huang, Yeyi Guan, Jinxiang Xia, Yi Yao, Hanhao Li, Ningning Wang, Pai Liu, Tianhao Peng, Xin Gui, Xiaowan Li, Yuhui Liu, Yuchen Eleanor Jiang, Jun Wang, Changwang Zhang, Xiangru Tang, Ge Zhang, Jian Yang, Minghao Liu, Xitong Gao, Jiaheng Liu, and Wangchunshu Zhou. Oagents: An empirical study of building effective agents, 2025. URL <https://arxiv.org/abs/2506.15741>.

A 更多生成支架的可视化

本附录在第 6.7 节两个代表性案例的基础上，补充展示同一四模块协议下生成的八个支架。这些样例覆盖约束密集型购物与旅行、分阶段 Web 产品构建、线索驱动研究、证据敏感检索、数值分析和工作区操作。它们并非单一执行模板的不同变体：JIT-Agent 会选择适合每项请求结构的状态表示和控制机制，再围绕该选择实例化规划、动作、记忆和能力编排。总体而言，这些案例让任务条件化支架定制在执行语义而非提示措辞层面变得可见。

JIT HARNESS ATLAS · 03

Origami — hierarchical work, folded context

Independent subtask memories preserve the work; a deliberate fold replaces only the active context.

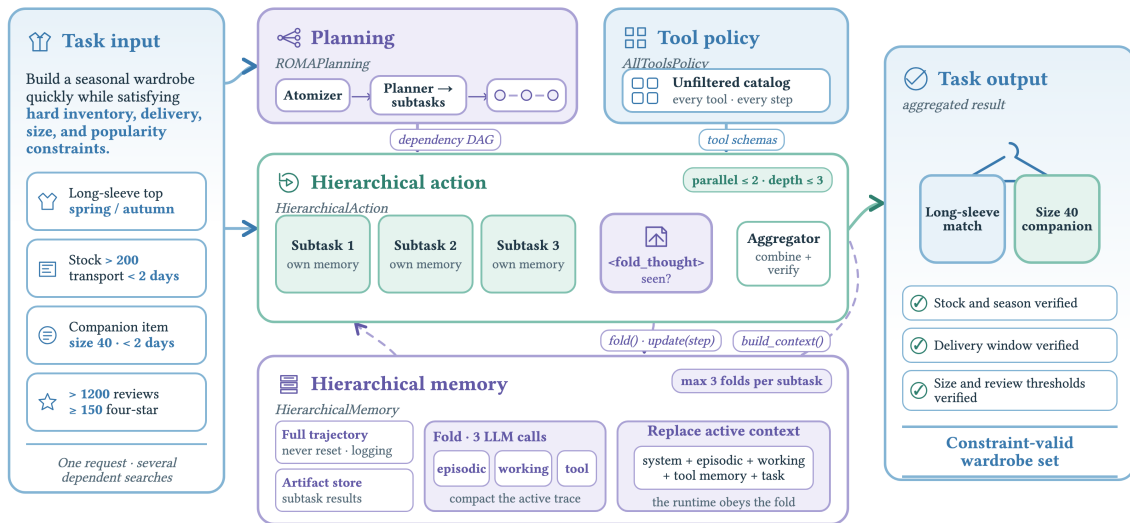


图 A.1: Origami: 使用折叠上下文的层次化工作。ROMAPPlanning 创建隔离的子任务；HierarchicalMemory 保留其轨迹和工件，而 fold_thought 在聚合前只替换活动工作上下文。

图 A.1 和图 A.2 展示了两种通过记忆控制长视野执行的任务专用方式。ORIGAMI 必须在库存、交付、尺码、评论数和评分等耦合约束下组合一套应季衣橱。ROMAPPlanning 将请求分解为相互依赖的商品搜索，HierarchicalAction 以有界深度并行运行至多两个分支，每个分支保留自己的轨迹和工件。当某个分支变长时，fold_thought 仅替换其活动工作上下文；已完成子任务的结果仍可用于最终约束检查。因此，该支架把上下文用在组合式购物任务中尚未解决的部分，同时不丢弃早先的商品证据。

相比之下，TURNSTILE 服务于一项明确日期、乘客、房间、出发时间窗、评分最高酒店、景点、餐食和每日路线的旅行请求。生成的 TravelPlanning 模块把这些义务编译为有类型的 travel_spec 和收集清单，DataStoreMemory 为每类证据分配专用存储桶。DynamicToolPolicy 暴露用于补齐缺失存储桶的搜索工具，但在 is_complete() 成功前不提供 final_answer。这里，记忆主要不是用于压缩长轨迹；它充当可执行的覆盖契约，防止系统在收集完所需旅行事实之前合成行程。

图 A.3 和图 A.4 展示了显式状态如何持续重塑后续决策。GEARBOX 为如下任务生成：在指定驾驶半径内寻找中餐馆，并将结果发布为交互式单页指南。其 PhaseAction 是共享搜索-收集-构建寄存器的唯一写入者；只有收集到足够多结构化餐馆记录后，系统才会转入构建阶段。PhaseToolPolicy 与 PhaseAwareMemory 都读取该寄存器，从 Web 发现切换到记录检查，再切换到编码与发布，同时把观察压缩为当前阶段所需的模式。由于任务形态从证据获取转为工件构建，模型由此创建了一台阶段机。

PEGBBOARD 则处理一个由六条人物传记与电影线索定义的身份识别问题。它把进度表示为候选项 × 线索矩阵，其中每条提取的主张都保留其文档标识符。空单元格和矛盾推动 DynamicResearchPlanning 开展定向搜索；当某一行获得足够支持时，则触发独立验证阶段，重新打开被引用的文档并解决冲

Turnstile — evidence first, itinerary second

A task-specific checklist withholds final_answer until every required travel fact has been collected.

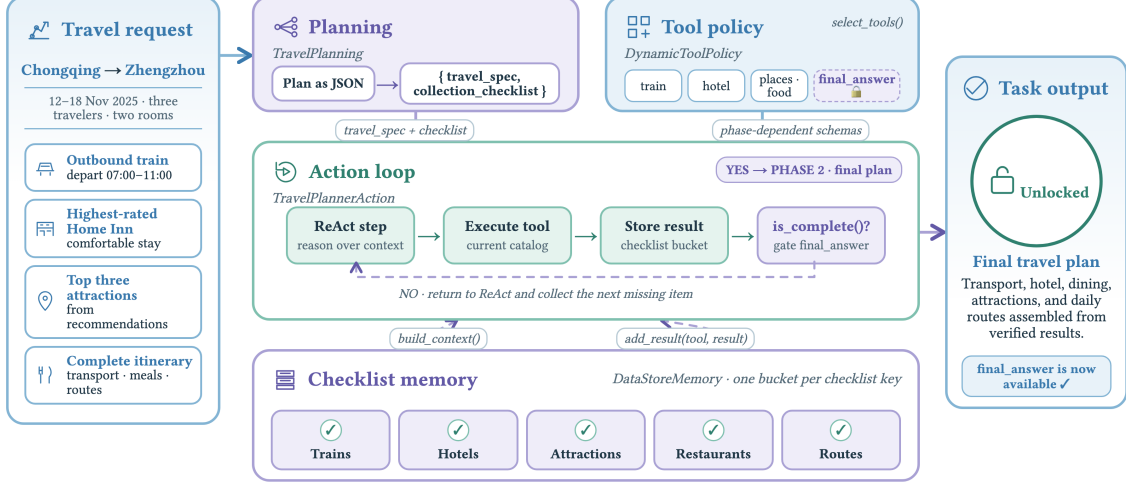


图 A.2: Turnstile: 证据优先, 行程随后。TravelPlanning 发出旅行规范和检查清单, DataStoreMemory 跟踪所需证据桶, 而 DynamicToolPolicy 只有在 is_complete() 成功后才暴露 final_answer。

Gearbox — one phase shift rewires the agent

The shared register switches tools and memory schemas across search → collect → build.

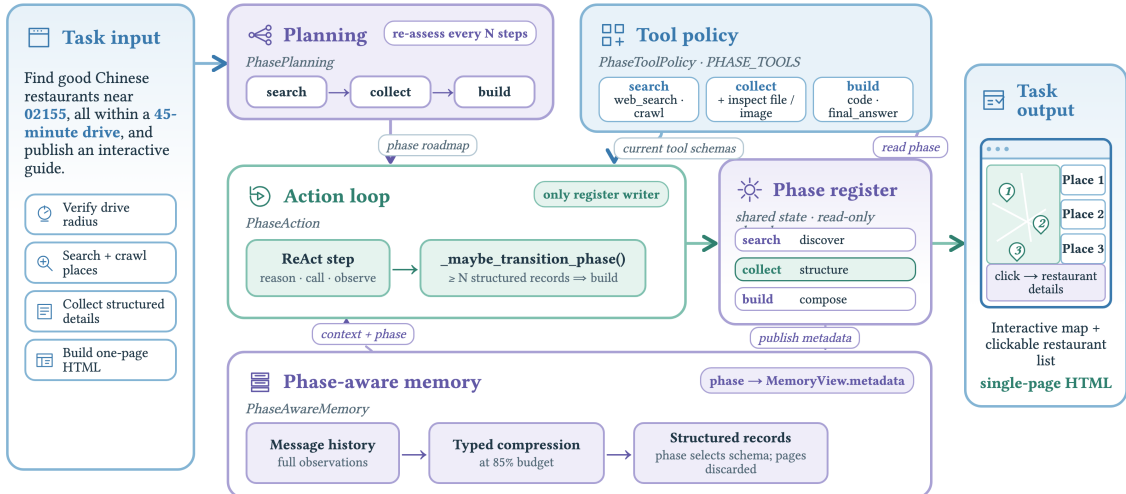


图 A.3: Gearbox: 一次阶段转换即可重写智能体。PhaseAction 是共享阶段寄存器的唯一写入者; PhaseToolPolicy 和 PhaseAwareMemory 读取该状态, 同时切换所暴露的能力与有类型的记忆模式。

Pegboard — research as an evidence matrix

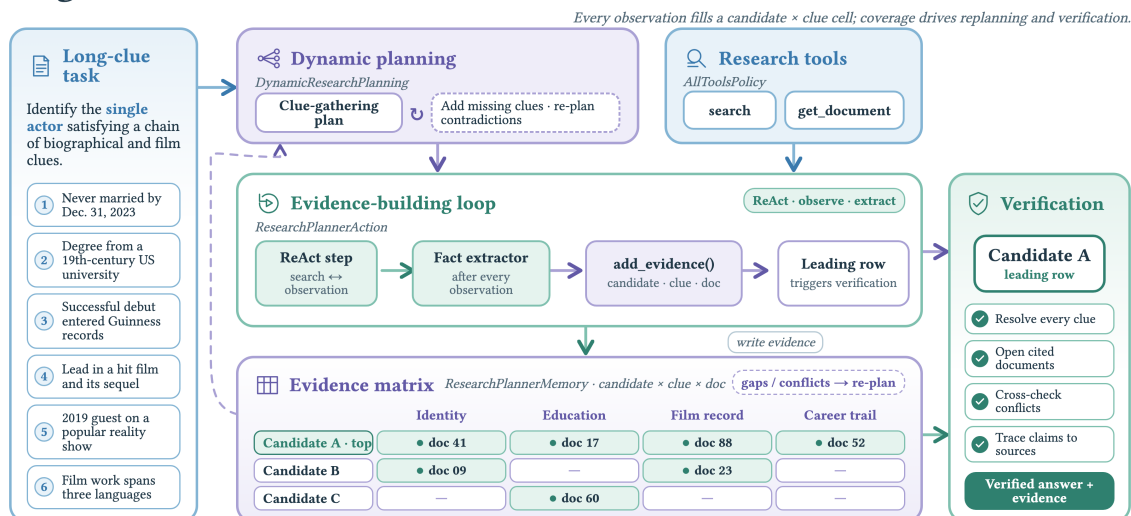


图 A.4: Pegboard: 把研究表示为证据矩阵。每个观察都被提取到带文档标识符的候选项 × 线索单元格中; 矩阵覆盖度同时驱动 DynamicResearchPlanning 和向基于来源的验证阶段转换。

突。不同于一般研究记录, 这种状态让覆盖度与来源溯源直接转化为可执行信号。与 GEARBOX 的对比表明, 生成状态既可以是小型阶段变量, 也可以是按任务塑形的证据结构, 取决于需要用什麼来支配下一动作。

Appraiser — evidence decides what the model sees

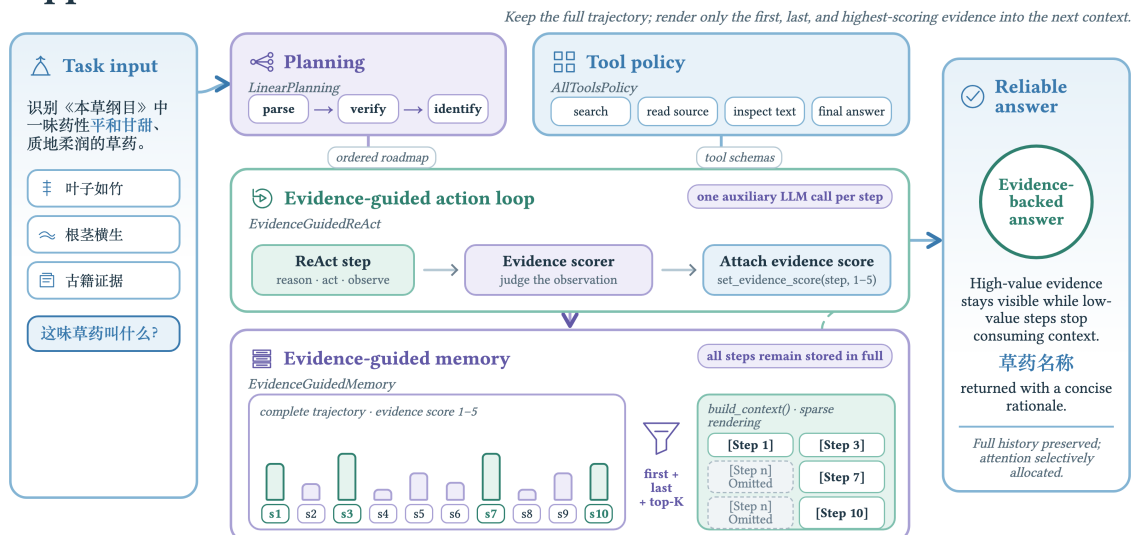


图 A.5: Appraiser: 证据决定模型看到什么。EvidenceGuidedReAct 为每一步评分; EvidenceGuidedMemory 存储完整运行过程, 但只把最初、最后和得分最高的 K 个观察渲染进 build_context()。

图 A.5 和图 A.6 定制后续推理从既往执行中接收的内容。APPRAISER 通过协调植物学特征与文本证据, 回答一个经典本草身份识别问题。每次搜索或阅读后, 辅助评分器都会为观察赋予证据价值;

Abacus — from computation to reliable typed state

Shell-derived evidence is parsed once, preserved structurally, and reported without re-deriving the numbers.

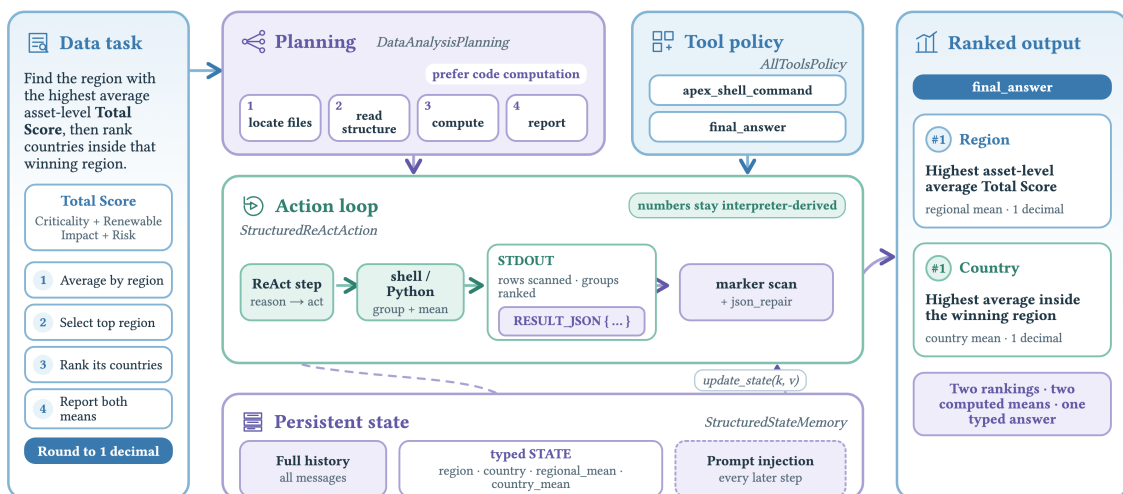


图 A.6: Abacus: 从计算走向可靠的有类型状态。StructuredReActAction 执行 shell 或 Python 代码，从标准输出中提取 RESULT_JSON 载荷，并更新 StructuredStateMemory，无需模型重新推导计算值。

EvidenceGuidedMemory 为可审计性保留完整轨迹，但只把最初、最后和得分最高的 K 个观察渲染到下一上下文。由此，低价值探索不再与最有助于识别的段落争夺上下文，同时没有证据从持久历史中被抹除。该支架专门适配这样一种检索任务：其主要瓶颈是证据显著性，而非工具选择。

ABACUS 面向一项数值分析请求生成：先比较各地区平均值，再对胜出地区内的国家排序。该支架有意将聚合移出语言模型推理：StructuredReActAction 调用 shell 或 Python，从标准输出中提取带标记的 RESULT_JSON 载荷，必要时修复其语法，并把数值写入 StructuredStateMemory。随后，地区、国家、地区均值和国家均值会作为有类型状态注入每个后续步骤，使最终答案报告解释器计算得到的量，而非从文字中重新推导。两个案例都保留完整审计轨迹，但都围绕自身任务占主导的可靠性要求渲染工作视图：一个重证据选择，另一个重数值保真。

图 A.7 和图 A.8 通过控制动作何时进入持久历史，为不同规模的工作区执行提供专用机制。PLAYER PIANO 处理配置文件的批量迁移，包括数据库、日志和 API 端点的更改。StructuredPlanning 发出有类型的发现、处理文件和最终回答步骤，由 Python 调度器执行；每次编辑后，确定性的非大语言模型检查会决定继续还是重试。ProgressiveTaskMemory 把逐文件看板渲染进系统提示，并同时保留原文件和已接受更改，防止大型批处理遗忘哪些文件确实通过了验证。

MULLIGAN 为一项更短的文档任务生成：从 DOCX 文件中提取学生标识符，并重命名同一文件。其线性计划无需层次结构或上下文压缩，因此支架保留完整的成功历史，只暴露 Word 与文件操作，同时隐藏无关的电子邮件、电子表格和 PDF 工具。若动作失败，循环会在本地追加错误，并在同一步骤重新生成，最多尝试两次；计划只有在提交成功动作后才会继续。因此，两个支架都关注可靠的工作区修改，但实现方式不同：一个使用有类型的批处理进度与逐文件验证，另一个使用窄工具暴露与事务式步骤修复。这种依赖任务的差异正是 JIT-Agent 所追求的行为：可靠性机制根据请求结构与故障面生成，而不是以同一个通用脚手架强加给所有任务。

Player Piano – typed batch editing, verified file by file

A Python interpreter executes a structured plan; deterministic checks control every transition.

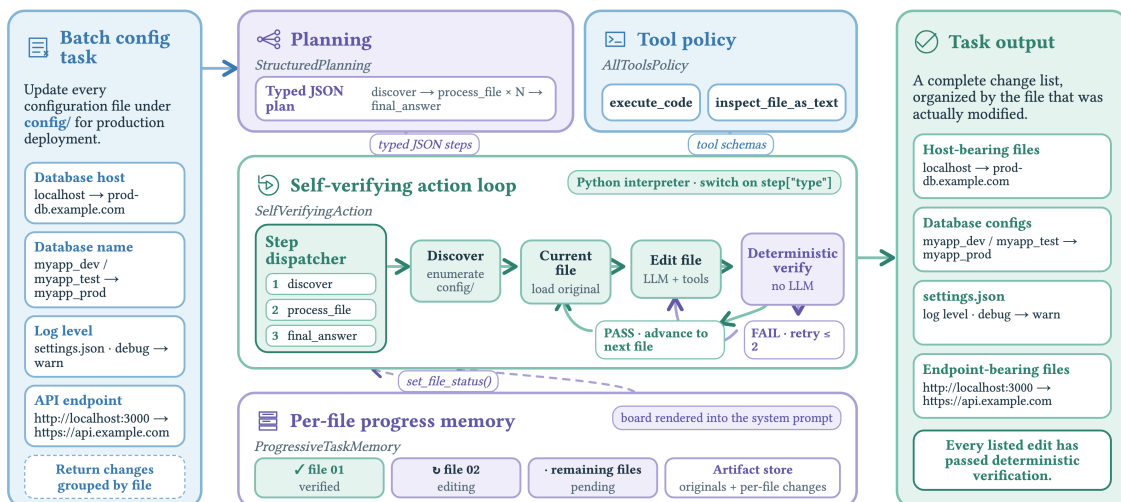


图 A.7: Player Piano: 有类型的批量编辑, 逐文件验证。Python 调度器解释 StructuredPlanning 步骤, SelfVerifyingAction 无需大语言模型即可检查每次编辑, 而 ProgressiveTaskMemory 维护逐文件看板和工作件存储。

Mulligan – repair the step, preserve the history

A failed action is regenerated in place; only successful repairs enter the transcript.

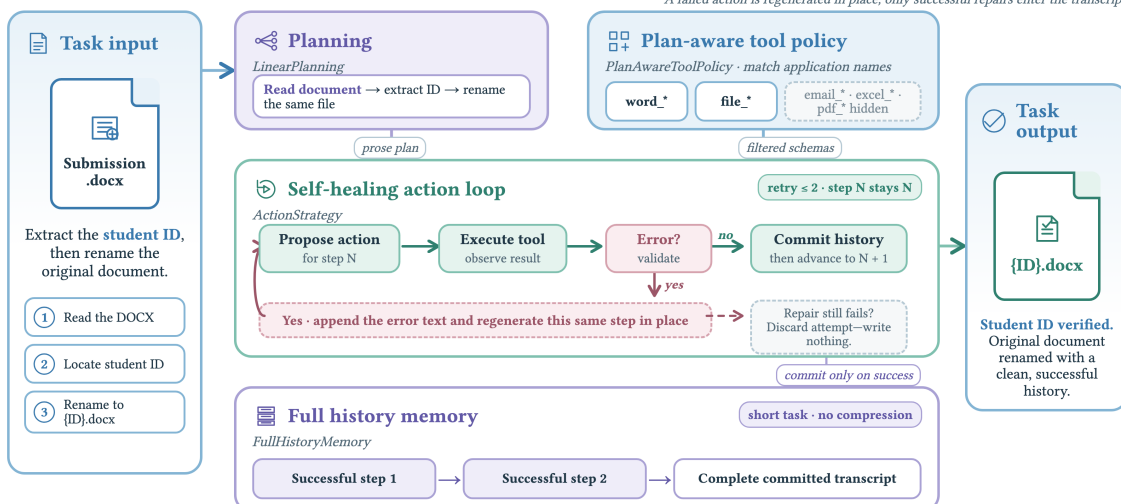


图 A.8: Mulligan: 修复步骤, 保留历史。PlanAwareToolPolicy 只暴露与计划相关的文件和文档工具; 失败动作在同一个步骤重新生成, 而 FullHistoryMemory 只接收成功执行。